KmerLife

Repository Bootstrap and Project Specification

Handoff document for the Antigravity bootstrap agent and the six-person development team

Item	Detail
Version	1.0
Date	7 October 2026
Purpose	Everything an AI agent needs to create, verify, push and organize the KmerLife GitHub repository, plus the game design, architecture, economy, team profiles and roadmap.
Audience	The Antigravity agent that bootstraps the repository; DEV-01 to DEV-06
Verification	All 48 repository files in Part 8 were installed from scratch in a clean folder: lint and typecheck are clean, 19 tests pass, and the economy simulator runs. The GitHub seeding script was dry-run. The live GitHub steps could not be run against a real account while preparing this document.

Document map

Part	Title	What it contains
1	Mission and rules of engagement	What the agent must do and must never do
2	Inputs to collect from the user	The only questions the agent may ask
3	Execution runbook	Step-by-step commands, with expected results
4	Game design	Vision, loop, scope, jobs, housing, njangi, events, seasons, legal
5	Architecture and economy	Principles, data flow, data model, security, tuning
6	Team and dev profiles	The six developers, what each owns, rules for their AI IDE
7	Roadmap and issue backlog	Six weeks, 26 issues, risks
8	Repository file manifest	Folder tree and the exact contents of all 48 files
9	Verification checklist and final report	How to prove the job is done
10	Troubleshooting, decisions and open questions	Known problems and what is still undecided

How to read the file blocks in Part 8

Each file has a heading with its path, then a line **BEGIN FILE: path**, the content in a grey block, and a line **END FILE: path**.

The file content is only what lies between the two marker lines. The markers themselves are not part of the file.

Preserve indentation and line breaks exactly. End every file with a single trailing newline.

If this document reaches you with the grey blocks or tables flattened and unreadable, ask the user to export it as Markdown or PDF before you start.

Part 1. Mission and rules of engagement

Mission

You are the bootstrap agent for **KmerLife**, a real-time multiplayer browser life-simulation game set in Cameroon. Your job has five parts:

1. Create the repository on the user's computer exactly as specified in Part 8.
2. Prove it works (lint, typecheck, tests, simulator) before anything leaves the machine.
3. Create the GitHub repository and push the first commit.
4. Seed GitHub with labels, milestones, 26 issues and branch protection using the script already in the repository.
5. Verify everything with the checklist in Part 9 and give the user a short report.

Hard rules

- **Follow this document literally.** Do not rename, reorder, "improve" or add files. Do not change any value in `src/game/constants.ts` or any test expectation to make a check pass.
- **Never ask for, accept, print or store a password or token.** The user authenticates by running `gh auth login` themselves.
- **Never commit secrets.** Only `.env.example` (empty values) may be committed.
- **No destructive git actions.** No force-push, no deleting remote branches, no changing repository visibility without the user's explicit yes.
- **Do not scaffold Next.js, Supabase or any other tool.** That is issue 1 and 2 for DEV-04. Do not add dependencies beyond those in `package.json`.
- **Ask only the questions in Part 2**, once, in a single message.
- **If a step fails twice, stop.** Report the exact command, the exact error, and what you tried. Do not work around it silently.
- **Work cross-platform.** The user may be on Windows. Use `git`, `node`, `npm` and `gh` commands only; avoid shell-specific syntax such as `&&` chaining.
- **Report honestly.** If something could not be done (for example branch protection on a free private repository), say so plainly.

Part 2. Inputs to collect from the user

Ask these in one message before touching anything. Use the defaults if the user says "you choose".

#	Question	Default	Used for
1	GitHub owner (user or organization) and repository name	Authenticated user's login, and `kmerlife`	Step 7 and the `--repo OWNER/NAME` argument
2	Visibility	private	Step 7
3	GitHub usernames of DEV-01 to DEV-06	The owner's own username for all six (user updates `.github/CODEOWNERS` later)	Step 4 (CODEOWNERS)
4	Invite the other developers as collaborators with Write access now?	No	Step 7b. CODEOWNERS entries only work for people with Write access
5	Local folder where `kmerlife` should be created	The current working folder	Step 2

Why the usernames matter
GitHub ignores CODEOWNERS lines whose owner is not a valid user with Write access to the repository. With branch protection "require review from code owners" this can block merges. If the user cannot provide usernames yet, use the owner's username everywhere and tell them to edit `.github/CODEOWNERS` once teammates are invited.

Part 3. Execution runbook

Run the steps in order. Each step lists what must be true before you continue. Replace `OWNER` with the answer to question 1.

Step 0. Check prerequisites

All four tools must exist, Node must be version 20 or newer, and the GitHub CLI must be logged in.

```
git --version
node -v
npm -v
gh --version
gh auth status
```

> **Expected**
> Versions print; `gh auth status` says you are logged in. If `gh` is missing, tell the user to install it from cli.github.com and run `gh auth login`, then stop. If Node is older than 20, stop and ask the user to upgrade.

Step 1. Ask the user the Part 2 questions

Send one message with the five questions and their defaults. Wait for the answers.

> **Expected**
> You now know OWNER, the repository name, visibility, six handles, whether to invite collaborators, and the local folder.

Step 2. Create the folder and initialize git

Create the `kmerlife` folder in the chosen location and initialize a repository with `main` as the default branch.

```
mkdir kmerlife
cd kmerlife
git init -b main
```

> **Expected**
> An empty git repository on branch `main`.

Step 3. Create every file from Part 8

Create all 48 files listed in Part 8 with exactly the specified content, including the six empty `.gitkeep` files. Create parent folders as needed.

Do not create `package-lock.json` or `node_modules` by hand; step 5 generates them.

> **Expected**
> 48 files exist. Nothing else exists except `.git`.

Step 4. Fill in CODEOWNERS

In `.github/CODEOWNERS` replace `@DEV01_HANDLE` through `@DEV06_HANDLE` with the real usernames from question 3 (each keeps its leading @). Then confirm no placeholder is left.

```
git grep -n "_HANDLE" -- .github/CODEOWNERS
```

> **Expected**
> The command prints nothing. (If git does not track the file yet, run `git add -A` first.)

Step 5. Install and prove the code works

Install dependencies (this creates `package-lock.json`), then run every check. All must pass before you commit.

```
npm install
```

```
npm run lint
npm run typecheck
npm test
npm run simulate -- --players 200 --days 30 --seed 1
```

Expected
Lint and typecheck print nothing after the script banner. Tests: 2 files, 19 tests, all passing. The simulator prints a table with four rows (grinder, regular, casual, spender) and no error. If something fails, the likely cause is a transcription error: compare the file with Part 8. Never edit constants or tests to force green.

Step 6. Make the first commit

Check that no secret file would be committed, then commit everything including `package-lock.json`.

```
git add -A
git ls-files .env*
git commit -m "chore: bootstrap KmerLife repository"
```

Expected
`git ls-files .env*` lists only `.env.example`.

Step 7. Create the GitHub repository and push

Create the remote from the local folder and push `main`. Use `--public` only if the user explicitly chose public.

```
gh repo create OWNER/kmerlife --private --source=. --remote=origin --push
```

Expected
The repository exists on GitHub and `main` is pushed. The CI workflow starts automatically on that push.

Step 7b. Invite collaborators (only if the user said yes)

For each teammate username, grant Write access. They must accept the invitation before their CODEOWNERS entries become valid.

```
gh api -X PUT repos/OWNER/kmerlife/collaborators/USERNAME -f permission=push
```

Expected
One invitation sent per teammate. Tell the user that invitations must be accepted.

Step 8. Seed labels, milestones, issues and branch protection

First do a dry run and read the output. Then run it for real. The script is idempotent: it upserts labels and skips milestones and issues that already exist.

```
node scripts/bootstrap-github.mjs --repo OWNER/kmerlife --dry-run
node scripts/bootstrap-github.mjs --repo OWNER/kmerlife
```

Expected
11 labels, 6 milestones, 26 issues, then "branch protection applied". If the script exits with code 2 and prints BRANCH PROTECTION NOT APPLIED, GitHub refused it (private repositories on a free personal plan cannot use branch protection). Report this to the user with the options the script prints. Do not make the repository public on your own.

Step 9. Tidy repository settings

Make merged pull-request branches delete themselves.

```
gh repo edit OWNER/kmerlife --delete-branch-on-merge
```

Expected
No error.

Step 10. Watch CI turn green

Wait for the workflow started by the push in step 7.

```
gh run list --repo OWNER/kmerlife --limit 1
gh run watch --repo OWNER/kmerlife
```

> **Expected**
> The latest run of "CI" finishes with success. If it fails, read the log with `gh run view --log-failed`, fix only transcription errors, and push the fix to `main` (the initial commit is the only one allowed directly on `main`).

Step 11. Run the Part 9 checklist and report

Verify each item, then send the user the report template from Part 9.

> **Expected**
> Every item is checked, or the report says exactly which item failed and why.

Part 4. Game design

This part is the readable form of `docs/GAME_DESIGN.md`. The exact Markdown is in Part 8.

1. Vision

KmerLife is a browser life-simulation game set in Cameroon. Players create a character, earn FCFA, pay rent, join njangi groups and live in a shared Yaoundé (Douala and Buea come later). It is built for mobile phones, slow networks and short sessions, in French and English.

Design pillars

- **Authentically Cameroonian.** Njangi, radio trottoir, Eneo power cuts, benskin rides, concours. Not a reskin of another country's game.
- **Time is the currency of play.** Energy and rent run on real clocks, so a few minutes a day is enough.
- **The server decides everything.** Clients never edit money, energy or housing.
- **The city is never empty.** Echoes (replayed real-player sessions) keep venues alive.
- **Fair and bounded.** Seasons reset the economy, FCFA has no cash value, nothing can be bought for an advantage.

2. Core loop

1. Create a character: name, origin, city.
2. Take a job.
3. Work: spend energy, earn FCFA.
4. Eat: spend FCFA, regain energy.
5. Pay rent every week. Fall behind and arrears build up; fall far behind and you are evicted.
6. Grow: move to better housing, join a njangi, travel, socialize.
7. React to the world: rumors, power cuts, market swings.
8. At season end the economy resets and the hall of fame is recorded.

3. MVP scope

Feature	MVP	Later
City	Yaoundé	Douala, Buea
Careers	5	More careers with levels and requirements (concours, dossiers)
Venues	8	More per city
Housing	3 tiers	More tiers, furniture
Transport	On foot, benskin, taxi, car	Clando, intercity bus
Social	Venue presence, city chat	Friends, guilds
Njangi	Week 5	Variants
Events	Rumors, power cuts	Water cuts, rainy season, scams
Echoes	Week 5	Richer replay
Languages	French and English	Camfranglais flavor in event text

Out of scope: cash-out of FCFA, real-money purchases, NPC characters, voice chat, user-generated content.

4. Characters

- Name: 2 to 24 characters.
- Origin: **privileged** (starts with 300 000 FCFA and a rented room) or **struggling** (starts with 0 FCFA, staying with relatives).
- City: Yaoundé only in the MVP.
- No cosmetic customization in the MVP.

5. Jobs (MVP careers)

Id	English	French	Wage per shift (FCFA)	Energy per shift
benskin_rider	Benskin rider	Conducteur de moto-taxi	900	15
market_trader	Market trader	Commerçant(e) de marché	700	12
tailor	Tailor	Couturier(ère)	1 000	15
momo_agent	Mobile money agent	Agent de transfert d'argent mobile	650	10
private_tutor	Private tutor	Répétiteur(trice)	1 300	18

The authoritative values live in `src/game/constants.ts`. Later, careers get levels and entry requirements such as an exam (concours) or paperwork (dossier).

6. Energy and meals

- Energy ranges from 0 to 100.
- Base regeneration is 0.1 per minute (6 per hour, full in under 17 hours), multiplied by the housing factor (0.8, 1.0 or 1.2).
- Working spends the job's energy cost.
- A meal costs 1 500 FCFA and restores 25 energy. You cannot eat when energy is full.

7. Housing and rent

Tier	Id	Weekly rent (FCFA)	Deposit (FCFA)	Regeneration factor
0	relatives	0	0	0.8
1	chambre	10 000	20 000	1.0
2	studio	25 000	50 000	1.2

Rules:

- Rent is charged automatically each week from the balance. Any shortfall becomes **arrears**.
- Arrears of two weekly rents or more mean **eviction** to tier 0. The debt remains.
- You must clear arrears before moving.
- Moving costs the deposit, which is not refunded in the MVP.
- If a player is away for a long time, at most 8 weeks of rent are charged in one catch-up.

8. Venues (MVP, Yaoundé)

Mokolo Market, Central Market, Mfoundi Market, Bastos, Mvog-Mbi, Mvan Bus Station, Municipal Lake, Ngoa-Ekellé Campus. The data lives in `content/venues.yaounde.json`; names are message keys.

9. Travel

Each trip costs time and FCFA depending on mode (on foot, benskin, taxi, car) and distance. The numbers are set in constants when the travel module is built in week 3.

10. Social

- Presence per venue and chat per city, in real time.
- Report and mute are mandatory from the first release; chat is rate-limited.
- **Echoes:** anonymized recordings of real players' action sequences are replayed as dimmed, clearly labelled figures in quiet venues. They never affect real players' economy.

11. Njangi (rotating savings)

A njangi (tontine in French) is a group where everyone pays a fixed amount each round and one member takes the whole pot, in turn.

- Group size 3 to 12 players; fixed contribution and round length set at creation.
- The order of receiving is drawn at random when the group starts.
- Each round the server collects every contribution; the pot is paid to that round's beneficiary only when all contributions are in.
- A missed contribution gets a 24-hour grace period. After that the defaulter is removed, loses any claim to a future pot, and carries a visible "unreliable" mark for the season.
- A member who already took the pot and then defaults owes the shortfall as debt.
- Open questions for DEV-05 and DEV-01: group discovery, invitations, whether members may swap turns.

12. World events and radio trottoir

Event types: **rumor**, **power_cut** (Eneo outage), **market_fluctuation**, **rent_due**.

- Rumors are short texts that spread through the city. Each has a hidden truth flag. If true, it has an economic effect (for example, prices in a market rise for 24 hours). Acting on a false rumor costs the player.
- Power cuts stop work shifts in the affected city for a time.
- Event text is written in advance (generated offline, reviewed by a human), stored in `content/`, in French and English. There is no live text generation at runtime.

Content rules: no real people, real brands, politics, ethnic stereotypes, or references to the Anglophone crisis.

13. Seasons

- A season lasts about 90 days.
- At the end: balances, housing and jobs reset; a hall of fame (wealthiest, most reliable njangi member) is kept.
- Calendar-linked events: Youth Day (11 February), National Day (20 May), and the December trip to the village (descente au village), when players save for gifts.

14. Money, ads and legal

- FCFA is virtual: it cannot be bought, sold or cashed out. No randomized paid rewards.
- **Phygital ads:** local businesses can sponsor venues or billboards. Spending in the game can unlock real discount codes at the real business. Sponsors see redemption counts only. Sponsored content is clearly labelled.
- Privacy: collect the minimum. Email for login. Phone numbers only for opt-in reminders. Publish a privacy notice in French and English, and have the current Cameroonian data-protection and cybersecurity rules checked before launch.
- Decide a minimum player age before the pilot.

15. Languages

French and English are equal. Routes are prefixed `/fr` and `/en`. All text lives in `src/messages/en.json` and `fr.json`; content files store message keys only.

16. Success metrics

- Day-1 and day-7 retention of the pilot cohort.
- Sessions per player per day and actions per session.
- Median wealth against the targets in `docs/ECONOMY.md`.
- Share of players evicted in their first two weeks.
- Chat reports per 1 000 messages.

17. Glossary

Term	Meaning
FCFA	Central African CFA franc, the currency
Njangi / tontine	Rotating savings group
Benskin	Motorbike taxi
Clando	Unlicensed shared taxi
Radio trottoir	Street rumor network
Eneo	Electricity distributor (power cuts)
Concours	Competitive entrance exam
Feyman	Con artist
Descente au village	The year-end trip to one's home village
MoMo	Mobile money

Part 5. Architecture and economy

Readable forms of `docs/ARCHITECTURE.md` and `docs/ECONOMY.md`. The exact Markdown is in Part 8.

Architecture

1. Principles

1. **Pure rules kernel.** All game rules live in `src/game/` as pure functions: `applyAction(state, action, now)` returns a new state and events. No I/O, no clock, no randomness.
2. **Server-authoritative.** Only the server writes player state. Clients may predict, but the server's answer wins.
3. **Lazy time.** Energy regeneration and rent are computed from timestamps when a player acts or loads. No tick loop is needed for them.
4. **Idempotent actions.** Every action carries a client-generated UUID key. Repeating a key never repeats the effect.
5. **Slow multiplayer.** Real-time is limited to presence, chat and events. Everything else refreshes on a slow cadence.
6. **Vertical features.** Each developer owns a feature end to end (rules, UI, content). The shared kernel stays small and is owned by the lead.
7. **Cheap by design.** Free tiers, small bundles, cost alerts. Built for 3G and low-end Android.

2. Layers

```
Browser (Next.js, Zustand, local prediction with src/game)
   |  POST /api/game  { action }              ^  realtime: presence, chat, events
   v                                          |
Server  src/server/handle-action.ts  --  applyAction (src/game)  --  Postgres (Supabase)
```

3. One action, step by step

1. The client creates a UUID `key` and POSTs the action.
2. The server authenticates the player.
3. It opens a transaction and selects the player row `FOR UPDATE`.
4. If `action_log` already holds (player, key), it returns the stored result and stops.
5. It calls `applyAction(state, action, Date.now())`, using the server clock only.
6. On success it updates the player row (checking `version`), inserts `action_log` with the result, and inserts `transactions` rows for FCFA changes.
7. It commits and returns the new state and events.
8. If relevant, it publishes a realtime event.

4. Data model (outline)

All money columns are `bigint` FCFA. The kernel uses epoch milliseconds; the database uses `timestamptz`; a mapper in `src/server/` converts.

Table	Key columns
characters	id, user_id (unique), name, origin, city, fcfa, energy numeric(5,2), job_id, housing_tier, rent_arrears_fcfa, next_rent_due_at, last_accrued_at, season_id, version, created_at
action_log	id, player_id, key, action jsonb, result jsonb, created_at, UNIQUE (player_id, key)

Table	Key columns
transactions	id, player_id, delta_fcfa, reason, season_id, created_at
world_state	city, key, value jsonb, updated_at
njangi_groups, njangi_members, njangi_rounds	Added in week 5
echo_recordings	Added in week 5

Row-level security on every table: a player reads only their own rows; nothing in the browser can write money, energy or housing.

5. Realtime

- Channels: `city:{city}`, `venue:{venueId}`, and a private `player:{id}`.
- The browser gets a scoped token from `/api/ably-token`; the provider API key never leaves the server.
- All access goes through the `RealtimeTransport` interface in `interfaces.md`.

6. Scheduled work

Only for world events, season changes, reminders and cleanup. Use Supabase `pg_cron` or Vercel Cron (check current plan limits). Every job is idempotent.

7. Security

- Validate every request with the zod schemas in `src/game/schemas.ts`.
- Rate-limit per player.
- Service-role and provider keys are server-only; `.env` is never committed.
- Secrets scanning enabled on the repository.
- Automated tests prove one player cannot read or write another's data.

8. Environments

Local, Preview (one per pull request), Production. Variables are listed in `.env.example`.

9. Testing

- Unit tests for the kernel (`tests/game/`), including purity and idempotency.
- Property tests (week 1) for money and energy invariants.
- API tests for the action handler, including concurrent calls with the same key.
- RLS tests in CI against a local Supabase.
- The economy simulator runs as a smoke test in CI.

10. Performance and cost guardrails

- Initial JavaScript budget: aim under 150 KB gzipped; images lazy-loaded and small.
- Realtime message volume is monitored; alerts on every paid service.
- Offline-tolerant UI: queue actions and replay with the same keys.

11. Decision records

- **ADR-0001 Pure kernel.** Rules are pure functions shared by server, client and simulator. This makes them testable, predictable and safe for AI-assisted development.

- **ADR-0002 Realtime provider.** Ably is the team's choice and is the default, behind `RealtimeTransport`. Supabase Realtime is the documented alternative if cost or complexity argues for it.
- **ADR-0003 Reminders.** PWA web push plus a Telegram bot (free) rather than WhatsApp Business API (paid).
- **ADR-0004 Seasons.** The economy resets about every 90 days to control inflation and give late joiners a fair start.

Economy

1. Principles

- Money is virtual FCFA, always an integer, never negative.
- Faucets (wages, starting money) must be balanced by sinks (rent, meals, deposits, and later transport, upgrades, njangi stakes).
- No number is final until the simulator says it is fine.

2. Current parameters

These mirror `src/game/constants.ts`, which is the single source of truth.

Parameter	Value
Energy maximum	100
Energy regeneration (base)	0.1 per minute
Meal	1 500 FCFA for 25 energy
Starting FCFA	300 000 (privileged), 0 (struggling)
Rent interval	7 days
Eviction	Arrears of 2 weekly rents or more
Maximum rent catch-up	8 periods
Wages per shift	benskin 900, trader 700, tailor 1 000, MoMo agent 650, tutor 1 300
Energy per shift	benskin 15, trader 12, tailor 15, MoMo agent 10, tutor 18
Housing rent / deposit	chambre 10 000 / 20 000, studio 25 000 / 50 000

3. Baseline simulation

Command: `npm run simulate -- --players 400 --days 60 --seed 7`

Strategy	Players	p10 FCFA	Median	p90	Average per day
grinder (6 logins a day)	100	461 000	1 247 500	2 558 000	22 557
regular (2 logins a day)	100	214 600	520 000	932 600	9 724
casual (1 login a day)	100	336 000	390 000	610 000	7 881
spender (2 logins, eats often)	100	324 000	720 000	1 252 000	12 598

Reading: the faucet is larger than the sinks. A casual player nets about 55 000 FCFA a week against a rent of 10 000 to 25 000. Players hoard. More money sinks are required before the pilot (issue: "Design and add money sinks").

4. Targets (proposals for DEV-01 to confirm)

1. A player who works every day is never evicted.
2. A grinder earns at most about 3 times a casual player's weekly net.
3. Median wealth grows roughly linearly over a season, not exponentially.
4. A player can afford the studio deposit within about 4 weeks of steady play, but not in week 1.

5. How to tune

1. Change values in `src/game/constants.ts` only.
2. Run `npm run simulate` with several seeds.
3. Paste the output table into the pull request and into section 3 above.
4. Do not merge economy changes without that evidence.

6. Known simplifications

- Energy regeneration over a period uses the housing tier at the start of the period.
- Deposits are not refunded.
- Eating is close to break-even against wages by design: it buys time, not profit.
- The simulator's bots are simple; real players will be cleverer.

> **Honest status of the economy**
> The numbers are placeholders. The first simulation shows the faucet is larger than the sinks, so players hoard money. Issues 8 and 9 (simulator v2 and money sinks) exist to fix this before the pilot. Do not tune the numbers during bootstrap.

Part 6. Team and dev profiles

Six developers, each owning a feature end to end (rules, interface, content) rather than a technical layer. This keeps AI agents inside clear boundaries and avoids a big integration crunch late in the project. Boundaries are enforced by `.github/CODEOWNERS` and branch protection, not by trust.

ID	Domain	Responsible for
DEV-01	Lead, Kernel and Economy	Rules engine, constants, simulator, CI, docs, final merge authority
DEV-02	Character, Jobs and Housing	Creation flow, dashboard, jobs, housing and rent UI
DEV-03	City, Travel and Venues	Map, venues, travel
DEV-04	Platform	Next.js shell, Supabase, auth, deploy, secrets
DEV-05	Social and Realtime	Presence, chat, njangi, offline queue, reminders
DEV-06	World, Events and Seasons	Rumors, power cuts, echoes, seasons

> **How a developer uses a card**
> Load `devs/DEV-0X.md` into the AI IDE as persistent context. Start every prompt with `[DEV-0X] [DOMAIN: your-domain]`.
> If a request needs a change outside the card's **Owns** list, the agent stops and says "This belongs to DEV-0Y", with a proposed change written as a note.

DEV-01: Lead, Kernel and Economy

Identity

You are assisting DEV-01, the project lead and merge gatekeeper for KmerLife, a Cameroon-set multiplayer browser life-simulation game. You protect the integrity of the rules kernel and the economy.

Owns (you may edit)

- `/src/game/*.ts` (constants, types, schemas, accrue, engine, player, index)
- `/src/lib/**`, `/src/server/**` (shared with DEV-04)
- `/scripts/**` (economy simulator, GitHub bootstrap)
- `/tests/game/**`
- `/docs/**`, `/devs/**`, `/.github/**`
- `AGENTS.md`, `CONVENTIONS.md`, `interfaces.md`, `package.json`

Read only (import, never edit)

- Feature modules under `/src/game/modules/<feature>/` (you only register them in the engine after review)
- UI under `/src/features/**` and routes under `/src/app/**`
- `/supabase/**`

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Every game number lives in `src/game/constants.ts`. Any change needs `npm run simulate` output in the PR.
3. Keep `applyAction` pure: no I/O, no clock, no randomness, no mutation of inputs.

4. Money is an integer number of FCFA, never negative, never a float.
5. Contract changes are made in `interfaces.md` in the same PR and announced in the team chat.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- Tests cover the new rule, including failure paths
- Simulator output attached if the economy changed
- `interfaces.md` updated if a contract changed
- `npm run lint`, `npm run typecheck` and `npm test` pass

DEV-02: Character, Jobs and Housing

Identity

You are assisting DEV-02, owner of character creation, the dashboard, and the jobs and housing features of KmerLife.

Owns (you may edit)

- `/src/game/modules/jobs/**`, `/src/game/modules/housing/**`
- `/src/features/character/**`, `/src/features/dashboard/**`

Read only (import, never edit)

- `/src/game/*.ts` (kernel and constants: read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- Other features' folders

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Read prices, wages and tiers from `src/game/constants.ts`. Never hardcode a game number.
3. Show energy using `accrue` for local prediction, but the server response always replaces local state.
4. Every action carries a fresh UUID idempotency key.
5. All user-facing text comes from `src/messages/en.json` and `fr.json`. Mobile-first, 360px minimum width, light assets.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- UI works in both languages
- No game rule implemented in a component
- Tests for any new pure rule in your modules
- `npm run lint`, `npm run typecheck` and `npm test` pass

DEV-03: City, Travel and Venues

Identity

You are assisting DEV-03, owner of the city map, venues and travel for KmerLife (Yaoundé first).

Owns (you may edit)

- `/src/game/modules/travel/**`, `/src/game/modules/venues/**`
- `/src/features/city/**`
- `/content/venues*`

Read only (import, never edit)

- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- `/src/services/realtime/**` (use its hooks, never edit)

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Venue and travel numbers come from constants; venue names are message keys, never literal strings.
3. For live positions or presence use DEV-05's realtime hooks from `interfaces.md`.
4. Keep total map assets small (target under 300 KB); design for 3G and low-end Android.
5. Real place names only for public landmarks; no real businesses or people.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- Content validated by a test
- Works offline-tolerant (no blocking on realtime)

- Both languages present
- `npm run lint`, `npm run typecheck` and `npm test` pass

DEV-04: Platform: Auth, Database, Deploy

Identity

You are assisting DEV-04, owner of the platform: Next.js app shell, Supabase schema and auth, secrets and deployment for KmerLife.

Owns (you may edit)

- `/supabase/**` (migrations, RLS, seed)
- `/src/app/**` (routes are thin shells that import from `src/features/*`)
- `/src/server/**` (shared with DEV-01)
- `/.env.example`

Read only (import, never edit)

- `/src/game/**` (rules and constants)
- `/src/features/**` (feature UI)
- `/src/services/**`

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Every table has row-level security. Players can never write money, energy or housing directly; only the server action handler does, through `applyAction`.
3. Every migration is reversible and generates fresh types into `src/lib/database.types.ts`.
4. Secrets only in the hosting dashboards. The Supabase service-role key and the Ably key are server-only and never get a `NEXT_PUBLIC_` prefix.
5. `action_log` has UNIQUE (player_id, key) for idempotency.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- RLS tested
- `.env.example` updated for any new variable
- Cost alerts set on any new paid service
- `npm run lint`, `npm run typecheck` and `npm test` pass

DEV-05: Social and Realtime

Identity

You are assisting DEV-05, owner of presence, chat, njangi, offline queueing and reminders for KmerLife.

Owns (you may edit)

- `/src/game/modules/njangi/**`, `/src/game/modules/social/**`
- `/src/features/social/**`
- `/src/services/realtime/**`

Read only (import, never edit)

- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- Other features' folders

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Realtime sits behind the `RealtimeTransport` interface in `interfaces.md` so the provider (Ably by default) can be swapped.
3. Clients never see the provider API key. Tokens are scoped so a player can only subscribe to their own city.
4. Njangi money moves only on the server, through pure rules in `/src/game/modules/njangi/`, with tests for default and dropout.
5. Rate-limit chat per player. Report and mute are mandatory. Reminders are opt-in.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- Works on a slow, flaky connection
- Moderation basics present
- Tests for every njangi rule
- `npm run lint`, `npm run typecheck` and `npm test` pass

DEV-06: World, Events and Seasons

Identity

You are assisting DEV-06, owner of world events (radio trottoir rumors, Eneo power cuts), echoes and seasons for KmerLife. You are the only system actor: there are no NPC characters.

Owns (you may edit)

- `/src/game/modules/events/**`, `/src/game/modules/seasons/**`
- `/src/services/world/**`
- `/content/events*`

Read only (import, never edit)

- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- `/src/services/realtime/**` (publish through its interface)

Rules

1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Time-based effects are computed lazily from timestamps wherever possible. Scheduled jobs are only for generating world events, season changes and cleanup.
3. Every scheduled job is idempotent: running it twice has the same result as once.
4. Event text is hand-reviewed content in FR and EN. No real people, brands, politics, ethnic stereotypes or references to the Anglophone crisis.
5. Echoes are anonymized, clearly labelled in the UI, and never affect real players' economy.
6. TypeScript strict. No `any`, no `@ts-ignore`.

Boundary protocol

If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the proposed change as a short note for the owner. Do not make the edit.

Definition of done

- Job proven idempotent by a test
- Content reviewed in both languages
- Events published via DEV-05's interface
- `npm run lint`, `npm run typecheck` and `npm test` pass

Part 7. Roadmap and issue backlog

Readable form of `docs/ROADMAP.md`. The same 26 issues are defined in `scripts/issues.json` and created on GitHub by the bootstrap script.

Six weeks from empty repo to a campus pilot. Every week must end in something playable.

Week 1: Foundations

Issue	Owner
Scaffold Next.js (App Router) into the existing repo	DEV-04
Supabase project, migrations and RLS	DEV-04
Auth and create-character endpoint	DEV-04
Server action handler: lock, reduce, persist	DEV-01
Character creation UI (FR/EN)	DEV-02
Dashboard: balance, energy, job picker, work button	DEV-02
Kernel hardening: property tests	DEV-01

Week 2: Economy and Content

Issue	Owner
Economy simulator v2 and tuning pass	DEV-01
Design and add money sinks: transport, living costs, upgrades	DEV-01
Housing UI and rent arrears flow	DEV-02
Venue content schema and eight Yaoundé venues	DEV-03
Event content schema and first ten radio trottoir rumors	DEV-06

Week 3: City and Deploy

Issue	Owner
City map UI	DEV-03
Travel module: on foot, benskin, taxi, car	DEV-03
Deploy: Vercel plus Supabase, preview environments	DEV-04

Week 4: Multiplayer

Issue	Owner
Ably token endpoint and venue presence	DEV-05
City chat with report and mute	DEV-05
Offline action queue with retry	DEV-05
Reminders: Telegram bot and web push	DEV-05

Week 5: Culture and Pilot Prep

Issue	Owner

Issue	Owner
Njangi module	DEV-05
Radio trottoir events engine and Eneo power cuts	DEV-06
Echoes: record and replay	DEV-06
Pilot readiness: analytics, flags, privacy notice	DEV-01

Week 6: Pilot

Issue	Owner
Run the campus pilot and review the data	DEV-01
Season 0 reset tooling	DEV-06
Security review: RLS tests and secrets audit	DEV-04

Definition of done for the pilot

- A new player can sign up, create a character, work, eat, pay rent, chat in a venue and receive a reminder.
- The economy simulator shows healthy targets (see `docs/ECONOMY.md`).
- RLS and idempotency tests are green in CI.
- Privacy notice published in French and English.

Risks

Risk	Mitigation
No players	Closed campus pilot, echoes, seasons
Team burnout and exams	Shippable weeks; two people know every module
Scope creep	MVP scope table in `docs/GAME_DESIGN.md` is the contract
Exploits	Row lock, idempotency keys, zod validation, ledger, rate limits
Cost spikes	Free tiers, alerts, small bundles

Part 8. Repository file manifest

The repository has 48 files (plus the generated `package-lock.json`). First the folder tree, then every file in full.

```
kmerlife/
  .env.example  .gitattributes  .gitignore  AGENTS.md  CONVENTIONS.md  README.md
  eslint.config.mjs  interfaces.md  package.json  tsconfig.json  vitest.config.ts
  .github/
    CODEOWNERS  PULL REQUEST TEMPLATE.md
    ISSUE_TEMPLATE/  feature.md  bug.md
    workflows/  ci.yml
  content/    venues.yaounde.json
  devs/       README.md  DEV-01.md  DEV-02.md  DEV-03.md  DEV-04.md  DEV-05.md  DEV-06.md
  docs/       GAME DESIGN.md  ARCHITECTURE.md  ECONOMY.md  ROADMAP.md
  scripts/    simulate.ts  bootstrap-github.mjs  issues.json
  src/
    app/ (empty)  features/ (empty)  lib/ (empty)  server/ (empty)  services/ (empty)
    game/     constants.ts  types.ts  schemas.ts  accrue.ts  player.ts  engine.ts  index.ts
    messages/ en.json  fr.json
  supabase/migrations/ (empty)
  tests/game/ engine.test.ts  content.test.ts
  (package-lock.json is generated by npm install and committed)
```

> **Reminder**
> Content between BEGIN and END markers is the whole file. Do not include the markers. Keep indentation exactly. End each file with one newline.

Root

.gitignore

BEGIN FILE: .gitignore

```
node_modules/
dist/
.next/
coverage/
.env
.env.*
!.env.example
.DS_Store
*.log
.vercel/
supabase/.temp/
```

END FILE: .gitignore

.gitattributes

BEGIN FILE: .gitattributes

```
* text=auto eol=lf
```

END FILE: .gitattributes

.env.example

BEGIN FILE: .env.example

```
# Copy to .env.local and fill in. NEVER commit real values.
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
# Server-only secrets (never expose to the browser, never prefix with NEXT_PUBLIC_):
SUPABASE_SERVICE_ROLE_KEY=
ABLY_API_KEY=
TELEGRAM_BOT_TOKEN=
```

END FILE: .env.example

package.json

BEGIN FILE: package.json

```
{
  "name": "kmerlife",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "engines": {
    "node": ">=20"
  },
  "scripts": {
```

```
    "lint": "eslint .",
    "typecheck": "tsc --noEmit",
    "test": "vitest run",
    "simulate": "tsx scripts/simulate.ts",
    "bootstrap:github": "node scripts/bootstrap-github.mjs"
  },
  "dependencies": {
    "zod": "^4.6.5"
  },
  "devDependencies": {
    "@types/node": "^26.6.4",
    "eslint": "^10.12.0",
    "tsx": "^4.23.15",
    "typescript": "^6.0.3",
    "typescript-eslint": "^8.71.1",
    "vitest": "^5.0.3"
  }
}
```

END FILE: package.json

tsconfig.json

BEGIN FILE: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "noEmit": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "forceConsistentCasingInFileNames": true,
    "types": ["node"]
  },
  "include": ["src", "tests", "scripts", "vitest.config.ts"],
  "exclude": ["node_modules", "dist", ".next"]
}
```

END FILE: tsconfig.json

vitest.config.ts

BEGIN FILE: vitest.config.ts

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    include: ['tests/**/*.test.ts'],
    environment: 'node',
  },
});
```

END FILE: vitest.config.ts

eslint.config.mjs

BEGIN FILE: eslint.config.mjs

```
import tseslint from 'typescript-eslint';

export default tseslint.config(
  { ignores: ['node_modules/**', 'dist/**', '.next/**', 'coverage/**'] },
  ...tseslint.configs.recommended,
  {
    rules: {
      '@typescript-eslint/no-explicit-any': 'error',
      '@typescript-eslint/ban-ts-comment': 'error',
    },
  },
);
```

END FILE: eslint.config.mjs

README.md

BEGIN FILE: README.md

```
# KmerLife

A real-time multiplayer browser life-simulation game set in Cameroon. Players create characters,
work, eat, pay rent, join njangi groups and live in a shared city. Yaoundé first, then Douala and
Buea. French and English.
```

````
## Quick start
```
node -v        # 20 or newer
npm install
npm run lint && npm run typecheck && npm test
npm run simulate -- --players 400 --days 60 --seed 7
```

## The team
See `devs/README.md`. Six developers, each owning a feature end to end. Ownership is enforced by
`.github/CODEOWNERS`.

## How we work
1. Load your dev card (`devs/DEV-0X.md`) into your AI IDE as persistent context.
2. Start every prompt with `[DEV-0X] [DOMAIN: your-domain]`.
3. Edit only inside your owned paths. If you need something elsewhere, open an issue or a note for
the owner.
4. Every change is a pull request. CI must pass and a code owner must approve.
5. Game numbers live only in `src/game/constants.ts`. Economy changes need simulator output.

## Documents
- `docs/GAME DESIGN.md`: what we are building
- `docs/ARCHITECTURE.md`: how it is built
- `docs/ECONOMY.md`: the numbers and how to tune them
- `docs/ROADMAP.md`: six-week plan (also in GitHub issues and milestones)
- `AGENTS.md`, `CONVENTIONS.md`, `interfaces.md`: rules for humans and AI agents
````

END FILE: README.md

AGENTS.md

BEGIN FILE: AGENTS.md

````
# Agent Rules: KmerLife

These rules apply to every AI agent working in this repository.

1. Read `CONVENTIONS.md` and `interfaces.md` before generating code.
2. The developer's prompt starts with `[DEV-0X] [DOMAIN: ...]`. Load `devs/DEV-0X.md` and obey it.
3. Edit only paths the dev card lists under **Owns**. `.github/CODEOWNERS` enforces this at review
time.
4. If a task needs a change elsewhere, stop and say "This belongs to DEV-0Y", then write the
proposed change as a note. Do not edit it.
5. Game rules are pure functions in `src/game/`. No I/O, no clock, no randomness, no mutation of
inputs.
6. The server is authoritative. Clients never write money, energy, housing or job. Every action has
an idempotency key and is validated with the zod schemas.
7. Game numbers live only in `src/game/constants.ts` (owner: DEV-01). Never hardcode them.
8. No user-facing string in code. Use `src/messages/en.json` and `src/messages/fr.json`, always
both.
9. TypeScript strict. No `any`, no `@ts-ignore`.
10. Never commit secrets. Never add a dependency without approval in the team chat; changes to
`package.json` need DEV-01's review.
11. Before finishing, run `npm run lint`, `npm run typecheck` and `npm test`, and fix failures.
12. When unsure, ask. Do not invent requirements.
````

END FILE: AGENTS.md

CONVENTIONS.md

BEGIN FILE: CONVENTIONS.md

````
# Conventions

## Stack
- Next.js (App Router), React, TypeScript (strict), Tailwind CSS, Zustand
- Supabase (Postgres, Auth), Ably (realtime, behind an interface), Vercel (hosting)
- Vitest for tests, ESLint with typescript-eslint, zod for validation
- Node 20 or newer

## Folder structure
- `src/game/`: pure rules kernel. Top-level files are owned by DEV-01; feature rules go in
`src/game/modules/<feature>/`.
- `src/features/<feature>/`: feature UI, owned by that feature's dev.
- `src/app/`: Next.js routes. Thin shells that import from `src/features/*`. Owned by DEV-04.
- `src/server/`: the action handler and database mapping. DEV-01 and DEV-04.
- `src/services/`: realtime (DEV-05) and world (DEV-06).
- `src/lib/`: shared clients and generated types. DEV-01.
- `src/messages/`: `en.json` and `fr.json`, always updated together.
- `content/`: game data as JSON; names are message keys.
- `supabase/`: migrations and seed. DEV-04.
- `scripts/`: simulator and GitHub bootstrap. DEV-01.

## Naming
````

```
- Files and folders: `kebab-case`
- Components: `PascalCase`; functions and variables: `camelCase`; types: `PascalCase`
- Constants: `SCREAMING_SNAKE_CASE`
- Ids in content and constants: `snake case` for jobs, `kebab-case` for venues

## Rules
- Money is an integer number of FCFA. Never use floats for money.
- All times in the kernel are epoch milliseconds from the server clock.
- Feature modules are registered with the engine through a small PR reviewed by DEV-01.
- No new dependency without a vote in the team chat.
- Commits: short imperative subject. Branches: `dev-0X/short-topic`.
- Pull requests: use the template; one concern per PR.
- Secrets: only in the hosting dashboards and local `.env.local`. Server-only keys never get a
`NEXT_PUBLIC_` prefix.
```

END FILE: CONVENTIONS.md

interfaces.md

BEGIN FILE: interfaces.md

```
# Interfaces: the contract between domains

The owner of a contract may change it only by editing this file in the same pull request and
announcing it. Consumers never change it.

## Shared kernel (DEV-01), `src/game/`
```ts
applyAction(state: PlayerState, action: unknown, now: number): ActionResult
accrue(state: PlayerState, now: number): { state: PlayerState; events: GameEvent[] }
createPlayer(input: CreatePlayerInput, now: number): PlayerState
actionSchema, createPlayerSchema            // zod
CITIES, MVP_CITIES, ORIGINS, JOB_IDS, JOBS, HOUSING_TIERS, MEAL, ...   // constants
type PlayerState, GameAction, GameEvent, ActionResult, ErrorCode
```
A feature module adds its actions by extending `actionSchema` and the engine switch in a PR
reviewed by DEV-01.

Action API (DEV-01 and DEV-04), `src/server/`
```
POST /api/game
body:     GameAction              (includes the idempotency key)
success:  { ok: true,  state: PlayerState, events: GameEvent[] }
failure:  { ok: false, code: ErrorCode }
```
Repeating the same key returns the stored result unchanged.

Platform (DEV-04)
- Tables: `characters`, `action log`, `transactions`, `world state` (see `docs/ARCHITECTURE.md`).
- Generated types: `src/lib/database.types.ts`.
- Auth: the server resolves the current player from the session; clients never send a player id.

Realtime (DEV-05), `src/services/realtime/`
```ts
interface RealtimeTransport {
  subscribe(channel: string, onMessage: (m: unknown) => void): () => void
  publish(channel: string, message: unknown): Promise<void>
  presence(venueId: string): { enter(): void; leave(): void; members(): string[] }
}
// React hooks built on it:
useVenuePresence(venueId: string): string[]
useCityChat(city: City): ChatMessage[]
sendChat(city: City, text: string): Promise<void>
```
`/api/ably-token` issues tokens scoped to the player's own city.

World (DEV-06), `src/services/world/`
```ts
type WorldEvent = {
  id: string
  city: City
  type: 'rumor' | 'power cut' | 'market fluctuation' | 'rent due'
  messageKey: string
  isTrue?: boolean            // rumors only, never sent to clients until resolved
  effect?: { kind: string; magnitude: number }
  expiresAt: string
}
useWorldEvents(city: City): WorldEvent[]
runWorldJob(city: City): Promise<void>   // scheduled, server-only, idempotent
```
```

END FILE: interfaces.md

GitHub configuration

.github/CODEOWNERS

BEGIN FILE: .github/CODEOWNERS

```
# CODEOWNERS: the real enforcement of domain boundaries.
# Replace every placeholder handle below with the real GitHub username BEFORE the first commit
# (find and replace @DEV01 HANDLE etc.). Invalid handles make GitHub ignore the line.
# Later lines override earlier ones.

# Default: the lead reviews anything not listed below.
*                                       @DEV01_HANDLE

# DEV-01 Kernel and Economy
/src/game/*.ts                          @DEV01 HANDLE
/src/lib/                               @DEV01_HANDLE
/scripts/                               @DEV01 HANDLE
/tests/game/                            @DEV01 HANDLE
/docs/                                  @DEV01 HANDLE
/devs/                                  @DEV01 HANDLE
/.github/                               @DEV01_HANDLE
/AGENTS.md                              @DEV01 HANDLE
/CONVENTIONS.md                         @DEV01 HANDLE
/interfaces.md                          @DEV01 HANDLE
/package.json                           @DEV01_HANDLE

# Shared server layer: lead and platform dev
/src/server/                            @DEV01_HANDLE @DEV04_HANDLE

# DEV-02 Character, Jobs and Housing
/src/game/modules/jobs/                 @DEV02 HANDLE
/src/game/modules/housing/              @DEV02_HANDLE
/src/features/character/                @DEV02 HANDLE
/src/features/dashboard/                @DEV02 HANDLE

# DEV-03 City, Travel and Venues
/src/game/modules/travel/               @DEV03 HANDLE
/src/game/modules/venues/               @DEV03_HANDLE
/src/features/city/                     @DEV03 HANDLE
/content/venues*                        @DEV03_HANDLE

# DEV-04 Platform (auth, database, deploy, app shell)
/supabase/                              @DEV04 HANDLE
/src/app/                               @DEV04_HANDLE
/.env.example                           @DEV04 HANDLE

# DEV-05 Social and Realtime
/src/game/modules/njangi/               @DEV05 HANDLE
/src/game/modules/social/               @DEV05 HANDLE
/src/features/social/                   @DEV05_HANDLE
/src/services/realtime/                 @DEV05 HANDLE

# DEV-06 World, Events and Seasons
/src/game/modules/events/               @DEV06 HANDLE
/src/game/modules/seasons/              @DEV06 HANDLE
/src/services/world/                    @DEV06_HANDLE
/content/events*                        @DEV06_HANDLE
```

END FILE: .github/CODEOWNERS

.github/PULL_REQUEST_TEMPLATE.md

BEGIN FILE: .github/PULL_REQUEST_TEMPLATE.md

```
## What and why
<!-- One short paragraph. Link the issue: Closes #123 -->

## Owner and domain
- Dev: DEV-0
- Files touched are all inside my domain (see .github/CODEOWNERS): yes / no (explain)

## Checklist
- [ ] `npm run lint`, `npm run typecheck`, `npm test` pass locally
- [ ] No hardcoded game numbers (everything from `src/game/constants.ts`)
- [ ] No user-facing strings in code (messages in `src/messages/en.json` and `fr.json`)
- [ ] If I changed a contract, I updated `interfaces.md` in this PR
- [ ] If I touched the economy, I attached `npm run simulate` output
```

END FILE: .github/PULL_REQUEST_TEMPLATE.md

.github/ISSUE_TEMPLATE/feature.md

BEGIN FILE: .github/ISSUE_TEMPLATE/feature.md

```
---
name: Feature
about: A new piece of functionality
labels: type:feature
---

**Owner:** DEV-0_

**Goal:**

**Acceptance criteria:**
-
```

END FILE: .github/ISSUE_TEMPLATE/feature.md

.github/ISSUE_TEMPLATE/bug.md

BEGIN FILE: .github/ISSUE_TEMPLATE/bug.md

```
---
name: Bug
about: Something is broken
labels: type:bug
---

**What happened:**

**What should have happened:**

**Steps to reproduce:**
1.

**Device, browser and network (e.g. Android Chrome, 3G):**
```

END FILE: .github/ISSUE_TEMPLATE/bug.md

.github/workflows/ci.yml

BEGIN FILE: .github/workflows/ci.yml

```
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - name: Economy smoke test
        run: npm run simulate -- --players 200 --days 30 --seed 1
```

END FILE: .github/workflows/ci.yml

Dev cards

devs/README.md

BEGIN FILE: devs/README.md

```
# Dev Cards

Each developer loads their own card into their AI IDE (Antigravity, Cursor, Claude Code, ...) as
persistent context and starts every prompt with `[DEV-0X] [DOMAIN: <domain>]`. Ownership is
enforced by `.github/CODEOWNERS`, not by trust.

| ID | Domain | Responsible for | Card |
|----|--------|-----------------|------|
```

```
| DEV-01 | Lead, Kernel and Economy | Rules engine, constants, simulator, CI, docs, merges | [DEV-
01.md](./DEV-01.md) |
| DEV-02 | Character, Jobs and Housing | Creation flow, dashboard, jobs, housing and rent UI |
[DEV-02.md](./DEV-02.md) |
| DEV-03 | City, Travel and Venues | Map, venues, travel | [DEV-03.md](./DEV-03.md) |
| DEV-04 | Platform | Next.js shell, Supabase, auth, deploy, secrets | [DEV-04.md](./DEV-04.md) |
| DEV-05 | Social and Realtime | Presence, chat, njangi, offline queue, reminders | [DEV-
05.md](./DEV-05.md) |
| DEV-06 | World, Events and Seasons | Rumors, power cuts, echoes, seasons | [DEV-06.md](./DEV-
06.md) |

GitHub handles: see `.github/CODEOWNERS` (single source of truth).
```

END FILE: devs/README.md

devs/DEV-01.md

BEGIN FILE: devs/DEV-01.md

```
# DEV-01: Lead, Kernel and Economy

## Identity
You are assisting DEV-01, the project lead and merge gatekeeper for KmerLife, a Cameroon-set
multiplayer browser life-simulation game. You protect the integrity of the rules kernel and the
economy.

## Owns (you may edit)
- `/src/game/*.ts` (constants, types, schemas, accrue, engine, player, index)
- `/src/lib/**`, `/src/server/**` (shared with DEV-04)
- `/scripts/**` (economy simulator, GitHub bootstrap)
- `/tests/game/**`
- `/docs/**`, `/devs/**`, `/.github/**`
- `AGENTS.md`, `CONVENTIONS.md`, `interfaces.md`, `package.json`

## Read only (import, never edit)
- Feature modules under `/src/game/modules/<feature>/` (you only register them in the engine after
review)
- UI under `/src/features/**` and routes under `/src/app/**`
- `/supabase/**`

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Every game number lives in `src/game/constants.ts`. Any change needs `npm run simulate` output
in the PR.
3. Keep `applyAction` pure: no I/O, no clock, no randomness, no mutation of inputs.
4. Money is an integer number of FCFA, never negative, never a float.
5. Contract changes are made in `interfaces.md` in the same PR and announced in the team chat.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- Tests cover the new rule, including failure paths
- Simulator output attached if the economy changed
- `interfaces.md` updated if a contract changed
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-01.md

devs/DEV-02.md

BEGIN FILE: devs/DEV-02.md

```
# DEV-02: Character, Jobs and Housing

## Identity
You are assisting DEV-02, owner of character creation, the dashboard, and the jobs and housing
features of KmerLife.

## Owns (you may edit)
- `/src/game/modules/jobs/**`, `/src/game/modules/housing/**`
- `/src/features/character/**`, `/src/features/dashboard/**`

## Read only (import, never edit)
- `/src/game/*.ts` (kernel and constants: read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- Other features' folders

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Read prices, wages and tiers from `src/game/constants.ts`. Never hardcode a game number.
```

```
3. Show energy using `accrue` for local prediction, but the server response always replaces local
state.
4. Every action carries a fresh UUID idempotency key.
5. All user-facing text comes from `src/messages/en.json` and `fr.json`. Mobile-first, 360px
minimum width, light assets.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- UI works in both languages
- No game rule implemented in a component
- Tests for any new pure rule in your modules
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-02.md

devs/DEV-03.md

BEGIN FILE: devs/DEV-03.md

```
# DEV-03: City, Travel and Venues

## Identity
You are assisting DEV-03, owner of the city map, venues and travel for KmerLife (Yaoundé first).

## Owns (you may edit)
- `/src/game/modules/travel/**`, `/src/game/modules/venues/**`
- `/src/features/city/**`
- `/content/venues*`

## Read only (import, never edit)
- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- `/src/services/realtime/**` (use its hooks, never edit)

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Venue and travel numbers come from constants; venue names are message keys, never literal
strings.
3. For live positions or presence use DEV-05's realtime hooks from `interfaces.md`.
4. Keep total map assets small (target under 300 KB); design for 3G and low-end Android.
5. Real place names only for public landmarks; no real businesses or people.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- Content validated by a test
- Works offline-tolerant (no blocking on realtime)
- Both languages present
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-03.md

devs/DEV-04.md

BEGIN FILE: devs/DEV-04.md

```
# DEV-04: Platform: Auth, Database, Deploy

## Identity
You are assisting DEV-04, owner of the platform: Next.js app shell, Supabase schema and auth,
secrets and deployment for KmerLife.

## Owns (you may edit)
- `/supabase/**` (migrations, RLS, seed)
- `/src/app/**` (routes are thin shells that import from `src/features/*`)
- `/src/server/**` (shared with DEV-01)
- `/.env.example`

## Read only (import, never edit)
- `/src/game/**` (rules and constants)
- `/src/features/**` (feature UI)
- `/src/services/**`

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Every table has row-level security. Players can never write money, energy or housing directly;
only the server action handler does, through `applyAction`.
```

```
3. Every migration is reversible and generates fresh types into `src/lib/database.types.ts`.
4. Secrets only in the hosting dashboards. The Supabase service-role key and the Ably key are
server-only and never get a `NEXT_PUBLIC_` prefix.
5. `action log` has UNIQUE (player id, key) for idempotency.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- RLS tested
- `.env.example` updated for any new variable
- Cost alerts set on any new paid service
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-04.md

devs/DEV-05.md

BEGIN FILE: devs/DEV-05.md

```
# DEV-05: Social and Realtime

## Identity
You are assisting DEV-05, owner of presence, chat, njangi, offline queueing and reminders for
KmerLife.

## Owns (you may edit)
- `/src/game/modules/njangi/**`, `/src/game/modules/social/**`
- `/src/features/social/**`
- `/src/services/realtime/**`

## Read only (import, never edit)
- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- Other features' folders

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
2. Realtime sits behind the `RealtimeTransport` interface in `interfaces.md` so the provider (Ably
by default) can be swapped.
3. Clients never see the provider API key. Tokens are scoped so a player can only subscribe to
their own city.
4. Njangi money moves only on the server, through pure rules in `/src/game/modules/njangi/`, with
tests for default and dropout.
5. Rate-limit chat per player. Report and mute are mandatory. Reminders are opt-in.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- Works on a slow, flaky connection
- Moderation basics present
- Tests for every njangi rule
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-05.md

devs/DEV-06.md

BEGIN FILE: devs/DEV-06.md

```
# DEV-06: World, Events and Seasons

## Identity
You are assisting DEV-06, owner of world events (radio trottoir rumors, Eneo power cuts), echoes
and seasons for KmerLife. You are the only system actor: there are no NPC characters.

## Owns (you may edit)
- `/src/game/modules/events/**`, `/src/game/modules/seasons/**`
- `/src/services/world/**`
- `/content/events*`

## Read only (import, never edit)
- `/src/game/*.ts` (read and import only)
- `/src/app/**`, `/src/lib/**`, `/src/server/**`, `/supabase/**`
- `/src/services/realtime/**` (publish through its interface)

## Rules
1. Read `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` before generating code.
```

```
2. Time-based effects are computed lazily from timestamps wherever possible. Scheduled jobs are
only for generating world events, season changes and cleanup.
3. Every scheduled job is idempotent: running it twice has the same result as once.
4. Event text is hand-reviewed content in FR and EN. No real people, brands, politics, ethnic
stereotypes or references to the Anglophone crisis.
5. Echoes are anonymized, clearly labelled in the UI, and never affect real players' economy.
6. TypeScript strict. No `any`, no `@ts-ignore`.

## Boundary protocol
If a request needs a change outside **Owns**, stop. Say: "This belongs to DEV-0X." and write the
proposed change as a short note for the owner. Do not make the edit.

## Definition of done
- Job proven idempotent by a test
- Content reviewed in both languages
- Events published via DEV-05's interface
- `npm run lint`, `npm run typecheck` and `npm test` pass
```

END FILE: devs/DEV-06.md

Documentation

docs/GAME_DESIGN.md

BEGIN FILE: docs/GAME_DESIGN.md

```
# KmerLife: Game Design

## 1. Vision
KmerLife is a browser life-simulation game set in Cameroon. Players create a character, earn FCFA,
pay rent, join njangi groups and live in a shared Yaoundé (Douala and Buea come later). It is built
for mobile phones, slow networks and short sessions, in French and English.

### Design pillars
- **Authentically Cameroonian.** Njangi, radio trottoir, Eneo power cuts, benskin rides, concours.
Not a reskin of another country's game.
- **Time is the currency of play.** Energy and rent run on real clocks, so a few minutes a day is
enough.
- **The server decides everything.** Clients never edit money, energy or housing.
- **The city is never empty.** Echoes (replayed real-player sessions) keep venues alive.
- **Fair and bounded.** Seasons reset the economy, FCFA has no cash value, nothing can be bought
for an advantage.

## 2. Core loop
1. Create a character: name, origin, city.
2. Take a job.
3. Work: spend energy, earn FCFA.
4. Eat: spend FCFA, regain energy.
5. Pay rent every week. Fall behind and arrears build up; fall far behind and you are evicted.
6. Grow: move to better housing, join a njangi, travel, socialize.
7. React to the world: rumors, power cuts, market swings.
8. At season end the economy resets and the hall of fame is recorded.

## 3. MVP scope
| Feature | MVP | Later |
|---------|-----|-------|
| City | Yaoundé | Douala, Buea |
| Careers | 5 | More careers with levels and requirements (concours, dossiers) |
| Venues | 8 | More per city |
| Housing | 3 tiers | More tiers, furniture |
| Transport | On foot, benskin, taxi, car | Clando, intercity bus |
| Social | Venue presence, city chat | Friends, guilds |
| Njangi | Week 5 | Variants |
| Events | Rumors, power cuts | Water cuts, rainy season, scams |
| Echoes | Week 5 | Richer replay |
| Languages | French and English | Camfranglais flavor in event text |

Out of scope: cash-out of FCFA, real-money purchases, NPC characters, voice chat, user-generated
content.

## 4. Characters
- Name: 2 to 24 characters.
- Origin: **privileged** (starts with 300 000 FCFA and a rented room) or **struggling** (starts
with 0 FCFA, staying with relatives).
- City: Yaoundé only in the MVP.
- No cosmetic customization in the MVP.

## 5. Jobs (MVP careers)
| Id | English | French | Wage per shift (FCFA) | Energy per shift |
|----|---------|--------|-----------------------|------------------|
| benskin_rider | Benskin rider | Conducteur de moto-taxi | 900 | 15 |
| market_trader | Market trader | Commerçant(e) de marché | 700 | 12 |
```

```
| tailor | Tailor | Couturier(ère) | 1 000 | 15 |
| momo agent | Mobile money agent | Agent de transfert d'argent mobile | 650 | 10 |
| private_tutor | Private tutor | Répétiteur(trice) | 1 300 | 18 |

The authoritative values live in `src/game/constants.ts`. Later, careers get levels and entry
requirements such as an exam (concours) or paperwork (dossier).

## 6. Energy and meals
- Energy ranges from 0 to 100.
- Base regeneration is 0.1 per minute (6 per hour, full in under 17 hours), multiplied by the
housing factor (0.8, 1.0 or 1.2).
- Working spends the job's energy cost.
- A meal costs 1 500 FCFA and restores 25 energy. You cannot eat when energy is full.

## 7. Housing and rent
| Tier | Id | Weekly rent (FCFA) | Deposit (FCFA) | Regeneration factor |
|------|----|--------------------|----------------|---------------------|
| 0 | relatives | 0 | 0 | 0.8 |
| 1 | chambre | 10 000 | 20 000 | 1.0 |
| 2 | studio | 25 000 | 50 000 | 1.2 |

Rules:
- Rent is charged automatically each week from the balance. Any shortfall becomes **arrears**.
- Arrears of two weekly rents or more mean **eviction** to tier 0. The debt remains.
- You must clear arrears before moving.
- Moving costs the deposit, which is not refunded in the MVP.
- If a player is away for a long time, at most 8 weeks of rent are charged in one catch-up.

## 8. Venues (MVP, Yaoundé)
Mokolo Market, Central Market, Mfoundi Market, Bastos, Mvog-Mbi, Mvan Bus Station, Municipal Lake,
Ngoa-Ekellé Campus. The data lives in `content/venues.yaounde.json`; names are message keys.

## 9. Travel
Each trip costs time and FCFA depending on mode (on foot, benskin, taxi, car) and distance. The
numbers are set in constants when the travel module is built in week 3.

## 10. Social
- Presence per venue and chat per city, in real time.
- Report and mute are mandatory from the first release; chat is rate-limited.
- **Echoes:** anonymized recordings of real players' action sequences are replayed as dimmed,
clearly labelled figures in quiet venues. They never affect real players' economy.

## 11. Njangi (rotating savings)
A njangi (tontine in French) is a group where everyone pays a fixed amount each round and one
member takes the whole pot, in turn.
- Group size 3 to 12 players; fixed contribution and round length set at creation.
- The order of receiving is drawn at random when the group starts.
- Each round the server collects every contribution; the pot is paid to that round's beneficiary
only when all contributions are in.
- A missed contribution gets a 24-hour grace period. After that the defaulter is removed, loses any
claim to a future pot, and carries a visible "unreliable" mark for the season.
- A member who already took the pot and then defaults owes the shortfall as debt.
- Open questions for DEV-05 and DEV-01: group discovery, invitations, whether members may swap
turns.

## 12. World events and radio trottoir
Event types: **rumor**, **power_cut** (Eneo outage), **market_fluctuation**, **rent_due**.
- Rumors are short texts that spread through the city. Each has a hidden truth flag. If true, it
has an economic effect (for example, prices in a market rise for 24 hours). Acting on a false rumor
costs the player.
- Power cuts stop work shifts in the affected city for a time.
- Event text is written in advance (generated offline, reviewed by a human), stored in `content/`,
in French and English. There is no live text generation at runtime.

Content rules: no real people, real brands, politics, ethnic stereotypes, or references to the
Anglophone crisis.

## 13. Seasons
- A season lasts about 90 days.
- At the end: balances, housing and jobs reset; a hall of fame (wealthiest, most reliable njangi
member) is kept.
- Calendar-linked events: Youth Day (11 February), National Day (20 May), and the December trip to
the village (descente au village), when players save for gifts.

## 14. Money, ads and legal
- FCFA is virtual: it cannot be bought, sold or cashed out. No randomized paid rewards.
- **Phygital ads:** local businesses can sponsor venues or billboards. Spending in the game can
unlock real discount codes at the real business. Sponsors see redemption counts only. Sponsored
content is clearly labelled.
```

````
- Privacy: collect the minimum. Email for login. Phone numbers only for opt-in reminders. Publish a
privacy notice in French and English, and have the current Cameroonian data-protection and
cybersecurity rules checked before launch.
- Decide a minimum player age before the pilot.

## 15. Languages
French and English are equal. Routes are prefixed `/fr` and `/en`. All text lives in
`src/messages/en.json` and `fr.json`; content files store message keys only.

## 16. Success metrics
- Day-1 and day-7 retention of the pilot cohort.
- Sessions per player per day and actions per session.
- Median wealth against the targets in `docs/ECONOMY.md`.
- Share of players evicted in their first two weeks.
- Chat reports per 1 000 messages.

## 17. Glossary
| Term | Meaning |
|------|---------|
| FCFA | Central African CFA franc, the currency |
| Njangi / tontine | Rotating savings group |
| Benskin | Motorbike taxi |
| Clando | Unlicensed shared taxi |
| Radio trottoir | Street rumor network |
| Eneo | Electricity distributor (power cuts) |
| Concours | Competitive entrance exam |
| Feyman | Con artist |
| Descente au village | The year-end trip to one's home village |
| MoMo | Mobile money |
````

END FILE: docs/GAME_DESIGN.md

docs/ARCHITECTURE.md

BEGIN FILE: docs/ARCHITECTURE.md

````
# KmerLife: Architecture

## 1. Principles
1. **Pure rules kernel.** All game rules live in `src/game/` as pure functions: `applyAction(state,
action, now)` returns a new state and events. No I/O, no clock, no randomness.
2. **Server-authoritative.** Only the server writes player state. Clients may predict, but the
server's answer wins.
3. **Lazy time.** Energy regeneration and rent are computed from timestamps when a player acts or
loads. No tick loop is needed for them.
4. **Idempotent actions.** Every action carries a client-generated UUID key. Repeating a key never
repeats the effect.
5. **Slow multiplayer.** Real-time is limited to presence, chat and events. Everything else
refreshes on a slow cadence.
6. **Vertical features.** Each developer owns a feature end to end (rules, UI, content). The shared
kernel stays small and is owned by the lead.
7. **Cheap by design.** Free tiers, small bundles, cost alerts. Built for 3G and low-end Android.

## 2. Layers
```
Browser (Next.js, Zustand, local prediction with src/game)
   |  POST /api/game  { action }              ^  realtime: presence, chat, events
   v                                          |
Server  src/server/handle-action.ts  --  applyAction (src/game)  --  Postgres (Supabase)
```

## 3. One action, step by step
1. The client creates a UUID `key` and POSTs the action.
2. The server authenticates the player.
3. It opens a transaction and selects the player row `FOR UPDATE`.
4. If `action_log` already holds (player, key), it returns the stored result and stops.
5. It calls `applyAction(state, action, Date.now())`, using the server clock only.
6. On success it updates the player row (checking `version`), inserts `action log` with the result,
and inserts `transactions` rows for FCFA changes.
7. It commits and returns the new state and events.
8. If relevant, it publishes a realtime event.

## 4. Data model (outline)
All money columns are `bigint` FCFA. The kernel uses epoch milliseconds; the database uses
`timestamptz`; a mapper in `src/server/` converts.

| Table | Key columns |
|-------|-------------|
| characters | id, user id (unique), name, origin, city, fcfa, energy numeric(5,2), job id,
housing tier, rent arrears fcfa, next rent due at, last accrued at, season id, version, created at
|
| action_log | id, player_id, key, action jsonb, result jsonb, created_at, UNIQUE (player_id, key)
|
````

```
| transactions | id, player_id, delta_fcfa, reason, season_id, created_at |
| world state | city, key, value jsonb, updated at |
| njangi_groups, njangi_members, njangi_rounds | Added in week 5 |
| echo recordings | Added in week 5 |

Row-level security on every table: a player reads only their own rows; nothing in the browser can
write money, energy or housing.

## 5. Realtime
- Channels: `city:{city}`, `venue:{venueId}`, and a private `player:{id}`.
- The browser gets a scoped token from `/api/ably-token`; the provider API key never leaves the
server.
- All access goes through the `RealtimeTransport` interface in `interfaces.md`.

## 6. Scheduled work
Only for world events, season changes, reminders and cleanup. Use Supabase `pg_cron` or Vercel Cron
(check current plan limits). Every job is idempotent.

## 7. Security
- Validate every request with the zod schemas in `src/game/schemas.ts`.
- Rate-limit per player.
- Service-role and provider keys are server-only; `.env` is never committed.
- Secrets scanning enabled on the repository.
- Automated tests prove one player cannot read or write another's data.

## 8. Environments
Local, Preview (one per pull request), Production. Variables are listed in `.env.example`.

## 9. Testing
- Unit tests for the kernel (`tests/game/`), including purity and idempotency.
- Property tests (week 1) for money and energy invariants.
- API tests for the action handler, including concurrent calls with the same key.
- RLS tests in CI against a local Supabase.
- The economy simulator runs as a smoke test in CI.

## 10. Performance and cost guardrails
- Initial JavaScript budget: aim under 150 KB gzipped; images lazy-loaded and small.
- Realtime message volume is monitored; alerts on every paid service.
- Offline-tolerant UI: queue actions and replay with the same keys.

## 11. Decision records
- **ADR-0001 Pure kernel.** Rules are pure functions shared by server, client and simulator. This
makes them testable, predictable and safe for AI-assisted development.
- **ADR-0002 Realtime provider.** Ably is the team's choice and is the default, behind
`RealtimeTransport`. Supabase Realtime is the documented alternative if cost or complexity argues
for it.
- **ADR-0003 Reminders.** PWA web push plus a Telegram bot (free) rather than WhatsApp Business API
(paid).
- **ADR-0004 Seasons.** The economy resets about every 90 days to control inflation and give late
joiners a fair start.
```

END FILE: docs/ARCHITECTURE.md

docs/ECONOMY.md

BEGIN FILE: docs/ECONOMY.md

```
# KmerLife: Economy

## 1. Principles
- Money is virtual FCFA, always an integer, never negative.
- Faucets (wages, starting money) must be balanced by sinks (rent, meals, deposits, and later
transport, upgrades, njangi stakes).
- No number is final until the simulator says it is fine.

## 2. Current parameters
These mirror `src/game/constants.ts`, which is the single source of truth.

| Parameter | Value |
|-----------|-------|
| Energy maximum | 100 |
| Energy regeneration (base) | 0.1 per minute |
| Meal | 1 500 FCFA for 25 energy |
| Starting FCFA | 300 000 (privileged), 0 (struggling) |
| Rent interval | 7 days |
| Eviction | Arrears of 2 weekly rents or more |
| Maximum rent catch-up | 8 periods |
| Wages per shift | benskin 900, trader 700, tailor 1 000, MoMo agent 650, tutor 1 300 |
| Energy per shift | benskin 15, trader 12, tailor 15, MoMo agent 10, tutor 18 |
| Housing rent / deposit | chambre 10 000 / 20 000, studio 25 000 / 50 000 |

## 3. Baseline simulation
```

```
Command: `npm run simulate -- --players 400 --days 60 --seed 7`

| Strategy | Players | p10 FCFA | Median | p90 | Average per day |
|---|---|---|---|---|---|
| grinder (6 logins a day) | 100 | 461 000 | 1 247 500 | 2 558 000 | 22 557 |
| regular (2 logins a day) | 100 | 214 600 | 520 000 | 932 600 | 9 724 |
| casual (1 login a day) | 100 | 336 000 | 390 000 | 610 000 | 7 881 |
| spender (2 logins, eats often) | 100 | 324 000 | 720 000 | 1 252 000 | 12 598 |

**Reading:** the faucet is larger than the sinks. A casual player nets about 55 000 FCFA a week
against a rent of 10 000 to 25 000. Players hoard. More money sinks are required before the pilot
(issue: "Design and add money sinks").

## 4. Targets (proposals for DEV-01 to confirm)
1. A player who works every day is never evicted.
2. A grinder earns at most about 3 times a casual player's weekly net.
3. Median wealth grows roughly linearly over a season, not exponentially.
4. A player can afford the studio deposit within about 4 weeks of steady play, but not in week 1.

## 5. How to tune
1. Change values in `src/game/constants.ts` only.
2. Run `npm run simulate` with several seeds.
3. Paste the output table into the pull request and into section 3 above.
4. Do not merge economy changes without that evidence.

## 6. Known simplifications
- Energy regeneration over a period uses the housing tier at the start of the period.
- Deposits are not refunded.
- Eating is close to break-even against wages by design: it buys time, not profit.
- The simulator's bots are simple; real players will be cleverer.
```

END FILE: docs/ECONOMY.md

docs/ROADMAP.md

BEGIN FILE: docs/ROADMAP.md

```
# KmerLife: Roadmap

Six weeks from empty repo to a campus pilot. Every week must end in something playable.

## Week 1: Foundations
| Issue | Owner |
|---|---|
| Scaffold Next.js (App Router) into the existing repo | DEV-04 |
| Supabase project, migrations and RLS | DEV-04 |
| Auth and create-character endpoint | DEV-04 |
| Server action handler: lock, reduce, persist | DEV-01 |
| Character creation UI (FR/EN) | DEV-02 |
| Dashboard: balance, energy, job picker, work button | DEV-02 |
| Kernel hardening: property tests | DEV-01 |

## Week 2: Economy and Content
| Issue | Owner |
|---|---|
| Economy simulator v2 and tuning pass | DEV-01 |
| Design and add money sinks: transport, living costs, upgrades | DEV-01 |
| Housing UI and rent arrears flow | DEV-02 |
| Venue content schema and eight Yaoundé venues | DEV-03 |
| Event content schema and first ten radio trottoir rumors | DEV-06 |

## Week 3: City and Deploy
| Issue | Owner |
|---|---|
| City map UI | DEV-03 |
| Travel module: on foot, benskin, taxi, car | DEV-03 |
| Deploy: Vercel plus Supabase, preview environments | DEV-04 |

## Week 4: Multiplayer
| Issue | Owner |
|---|---|
| Ably token endpoint and venue presence | DEV-05 |
| City chat with report and mute | DEV-05 |
| Offline action queue with retry | DEV-05 |
| Reminders: Telegram bot and web push | DEV-05 |

## Week 5: Culture and Pilot Prep
| Issue | Owner |
|---|---|
| Njangi module | DEV-05 |
| Radio trottoir events engine and Eneo power cuts | DEV-06 |
| Echoes: record and replay | DEV-06 |
```

```
| Pilot readiness: analytics, flags, privacy notice | DEV-01 |

## Week 6: Pilot
| Issue | Owner |
|-------|-------|
| Run the campus pilot and review the data | DEV-01 |
| Season 0 reset tooling | DEV-06 |
| Security review: RLS tests and secrets audit | DEV-04 |

## Definition of done for the pilot
- A new player can sign up, create a character, work, eat, pay rent, chat in a venue and receive a
reminder.
- The economy simulator shows healthy targets (see `docs/ECONOMY.md`).
- RLS and idempotency tests are green in CI.
- Privacy notice published in French and English.

## Risks
| Risk | Mitigation |
|------|------------|
| No players | Closed campus pilot, echoes, seasons |
| Team burnout and exams | Shippable weeks; two people know every module |
| Scope creep | MVP scope table in `docs/GAME DESIGN.md` is the contract |
| Exploits | Row lock, idempotency keys, zod validation, ledger, rate limits |
| Cost spikes | Free tiers, alerts, small bundles |
```

END FILE: docs/ROADMAP.md

Rules kernel (src/game)

src/game/constants.ts

BEGIN FILE: src/game/constants.ts

```ts
/**
 * GAME CONSTANTS: the economy.
 * Owner: DEV-01 only (enforced by CODEOWNERS).
 *
 * Money is FCFA, always an integer. Durations are milliseconds.
 * Every number here is a PLACEHOLDER until validated with `npm run simulate`
 * (see docs/ECONOMY.md). Do not hardcode any of these values elsewhere.
 */

export const MS_PER_MINUTE = 60_000;
export const MS_PER_HOUR = 3_600_000;
export const MS_PER_DAY = 86_400_000;
export const MS_PER_WEEK = 604_800_000;

export const CITIES = ['yaounde', 'douala', 'buea'] as const;
export const MVP_CITIES = ['yaounde'] as const;
export const ORIGINS = ['privileged', 'struggling'] as const;

export const ENERGY_MAX = 100;
export const ENERGY_REGEN_PER_MIN = 0.1;
export const STARTING_ENERGY = 100;

export const MEAL = { costFcfa: 1_500, energy: 25 } as const;

export const STARTING_FCFA = { privileged: 300_000, struggling: 0 } as const;
export const STARTING_HOUSING_TIER = { privileged: 1, struggling: 0 } as const;

export const RENT_INTERVAL_MS = MS_PER_WEEK;
/** Max rent periods charged in one catch-up (a player away for months is not instantly ruined). */
export const MAX_RENT_CATCHUP = 8;
/** Arrears of this many weekly rents or more => eviction to tier 0. */
export const EVICTION_ARREARS_PERIODS = 2;

export const JOB_IDS = [
  'benskin_rider',
  'market_trader',
  'tailor',
  'momo_agent',
  'private_tutor',
] as const;

export interface JobDef {
  readonly wageFcfa: number;
  readonly energyCost: number;
}

export const JOBS: Readonly<Record<(typeof JOB_IDS)[number], JobDef>> = {
  benskin_rider: { wageFcfa: 900, energyCost: 15 },
```

```
  market_trader: { wageFcfa: 700, energyCost: 12 },
  tailor: { wageFcfa: 1 000, energyCost: 15 },
  momo_agent: { wageFcfa: 650, energyCost: 10 },
  private tutor: { wageFcfa: 1 300, energyCost: 18 },
};

export interface HousingDef {
  readonly id: string;
  readonly rentFcfa: number;
  readonly depositFcfa: number;
  /** Multiplier on energy regeneration while living here. */
  readonly regenMult: number;
}

/** Index = tier. Tier 0 is the fallback (staying with relatives), always free. */
export const HOUSING TIERS: readonly [HousingDef, HousingDef, HousingDef] = [
  { id: 'relatives', rentFcfa: 0, depositFcfa: 0, regenMult: 0.8 },
  { id: 'chambre', rentFcfa: 10 000, depositFcfa: 20 000, regenMult: 1.0 },
  { id: 'studio', rentFcfa: 25_000, depositFcfa: 50_000, regenMult: 1.2 },
];
```

END FILE: src/game/constants.ts

src/game/types.ts

BEGIN FILE: src/game/types.ts

```
import type { CITIES, JOB IDS, ORIGINS } from './constants';

export type City = (typeof CITIES)[number];
export type Origin = (typeof ORIGINS)[number];
export type JobId = (typeof JOB IDS)[number];
export type HousingTier = 0 | 1 | 2;

/** The full, authoritative state of one player. Persisted as one row; the server is the only
writer. */
export interface PlayerState {
  id: string;
  name: string;
  origin: Origin;
  city: City;
  /** Integer FCFA. */
  fcfa: number;
  /** 0..ENERGY_MAX, two decimals. */
  energy: number;
  jobId: JobId | null;
  housingTier: HousingTier;
  rentArrearsFcfa: number;
  /** Epoch ms. Ignored while housingTier is 0. */
  nextRentDueAt: number;
  /** Epoch ms of the last time-based accrual (energy, rent). */
  lastAccruedAt: number;
  seasonId: number;
  /** Increments on every successful action (optimistic concurrency / audit). */
  version: number;
}

/** `key` is a client-generated idempotency key (UUID). The server stores (playerId, key) uniquely.
*/
export type GameAction =
  | { type: 'take job'; key: string; jobId: JobId }
  | { type: 'work'; key: string }
  | { type: 'buy_meal'; key: string }
  | { type: 'pay arrears'; key: string; amountFcfa: number }
  | { type: 'move housing'; key: string; tier: HousingTier };

export type GameEvent =
  | { type: 'rent charged'; periods: number; paidFcfa: number; arrearsAddedFcfa: number }
  | { type: 'evicted' }
  | { type: 'job taken'; jobId: JobId }
  | { type: 'worked'; earnedFcfa: number; energySpent: number }
  | { type: 'meal bought'; costFcfa: number; energyGained: number }
  | { type: 'arrears paid'; paidFcfa: number; remainingFcfa: number }
  | { type: 'moved'; tier: HousingTier; depositFcfa: number };

export const ERROR CODES = [
  'INVALID ACTION',
  'NO_JOB',
  'SAME JOB',
  'NOT_ENOUGH_ENERGY',
  'NOT ENOUGH FCFA',
  'ENERGY_FULL',
  'NOTHING OWED',
  'HAS_ARREARS',
```

```
  'SAME_HOUSING',
] as const;

export type ErrorCode = (typeof ERROR_CODES)[number];

export type ActionResult =
  | { ok: true; state: PlayerState; events: GameEvent[] }
  | { ok: false; code: ErrorCode };
```

END FILE: src/game/types.ts

src/game/schemas.ts

BEGIN FILE: src/game/schemas.ts

```
import { z } from 'zod';
import { CITIES, JOB_IDS, ORIGINS } from './constants';

const key = z.string().min(8).max(100);
const tier = z.union([z.literal(0), z.literal(1), z.literal(2)]);

export const actionSchema = z.discriminatedUnion('type', [
  z.object({ type: z.literal('take job'), key, jobId: z.enum(JOB IDS) }),
  z.object({ type: z.literal('work'), key }),
  z.object({ type: z.literal('buy_meal'), key }),
  z.object({ type: z.literal('pay_arrears'), key, amountFcfa: z.number().int().positive() }),
  z.object({ type: z.literal('move housing'), key, tier }),
]);

export const createPlayerSchema = z.object({
  id: z.string().min(1),
  name: z.string().trim().min(2).max(24),
  origin: z.enum(ORIGINS),
  city: z.enum(CITIES),
});
```

END FILE: src/game/schemas.ts

src/game/accrue.ts

BEGIN FILE: src/game/accrue.ts

```
import {
  ENERGY_MAX,
  ENERGY_REGEN_PER_MIN,
  EVICTION_ARREARS_PERIODS,
  HOUSING_TIERS,
  MAX_RENT_CATCHUP,
  MS_PER_MINUTE,
  RENT_INTERVAL_MS,
} from './constants';
import type { GameEvent, PlayerState } from './types';

const round2 = (n: number): number => Math.round(n * 100) / 100;

/**
 * Lazy time accrual. Brings a player's state up to `now` by computing, from timestamps,
 * what the simulation "would have done": energy regeneration and rent charges.
 * Pure and idempotent: accrue(accrue(s, t), t) equals accrue(s, t).
 * No background tick is needed for any of this.
 */
export function accrue(
  state: PlayerState,
  now: number,
): { state: PlayerState; events: GameEvent[] } {
  if (now <= state.lastAccruedAt) return { state, events: [] };

  const events: GameEvent[] = [];
  const next: PlayerState = { ...state };
  const housing = HOUSING_TIERS[state.housingTier];

  // Energy
  const minutes = (now - state.lastAccruedAt) / MS_PER_MINUTE;
  const regenPerMin = ENERGY_REGEN_PER_MIN * housing.regenMult;
  next.energy = round2(Math.min(ENERGY_MAX, state.energy + minutes * regenPerMin));

  // Rent (auto-debit; any shortfall becomes arrears)
  if (housing.rentFcfa > 0 && now >= state.nextRentDueAt) {
    const periodsElapsed = Math.floor((now - state.nextRentDueAt) / RENT_INTERVAL_MS) + 1;
    const periodsCharged = Math.min(periodsElapsed, MAX_RENT_CATCHUP);
    const totalDue = periodsCharged * housing.rentFcfa;
    const paid = Math.min(next.fcfa, totalDue);
    const added = totalDue - paid;
    next.fcfa -= paid;
```

```
      next.rentArrearsFcfa += added;
      next.nextRentDueAt = state.nextRentDueAt + periodsElapsed * RENT_INTERVAL_MS;
      events.push({
        type: 'rent charged',
        periods: periodsCharged,
        paidFcfa: paid,
        arrearsAddedFcfa: added,
      });

      if (next.rentArrearsFcfa >= EVICTION_ARREARS_PERIODS * housing.rentFcfa) {
        next.housingTier = 0;
        events.push({ type: 'evicted' });
      }
    }

    next.lastAccruedAt = now;
    return { state: next, events };
  }
```

END FILE: src/game/accrue.ts

src/game/player.ts

BEGIN FILE: src/game/player.ts

```
  import {
    MVP_CITIES,
    RENT_INTERVAL_MS,
    STARTING_ENERGY,
    STARTING_FCFA,
    STARTING_HOUSING_TIER,
  } from './constants';
  import { createPlayerSchema } from './schemas';
  import type { City, Origin, PlayerState } from './types';

  export interface CreatePlayerInput {
    id: string;
    name: string;
    origin: Origin;
    city: City;
  }

  /**
   * Pure factory for a brand-new player. Throws on invalid input.
   * Only MVP cities are playable until the roadmap opens Douala and Buea.
   */
  export function createPlayer(input: CreatePlayerInput, now: number): PlayerState {
    const parsed = createPlayerSchema.parse(input);
    if (!(MVP_CITIES as readonly string[]).includes(parsed.city)) {
      throw new Error(`City not open yet: ${parsed.city}`);
    }
    const housingTier = STARTING_HOUSING_TIER[parsed.origin];
    return {
      id: parsed.id,
      name: parsed.name,
      origin: parsed.origin,
      city: parsed.city,
      fcfa: STARTING_FCFA[parsed.origin],
      energy: STARTING_ENERGY,
      jobId: null,
      housingTier,
      rentArrearsFcfa: 0,
      nextRentDueAt: now + RENT_INTERVAL_MS,
      lastAccruedAt: now,
      seasonId: 0,
      version: 0,
    };
  }
```

END FILE: src/game/player.ts

src/game/engine.ts

BEGIN FILE: src/game/engine.ts

```
  import { accrue } from './accrue';
  import {
    ENERGY_MAX,
    HOUSING_TIERS,
    JOBS,
    MEAL,
    RENT_INTERVAL_MS,
  } from './constants';
  import { actionSchema } from './schemas';
  import type { ActionResult, ErrorCode, GameEvent, PlayerState } from './types';
```

```
const fail = (code: ErrorCode): ActionResult => ({ ok: false, code });

/**
 * THE rules of the game as one pure function.
 * Given the current state, an action and the current time, return the new state and events,
 * or an error code. No I/O, no clock, no randomness, no mutation of the inputs.
 *
 * The server calls this inside a row lock and persists the result.
 * The client may call it for optimistic prediction; the server's answer always wins.
 */
export function applyAction(state: PlayerState, action: unknown, now: number): ActionResult {
  const parsed = actionSchema.safeParse(action);
  if (!parsed.success) return fail('INVALID_ACTION');
  const act = parsed.data;

  const accrued = accrue(state, now);
  const s: PlayerState = { ...accrued.state };
  const events: GameEvent[] = [...accrued.events];

  switch (act.type) {
    case 'take job': {
      if (s.jobId === act.jobId) return fail('SAME JOB');
      s.jobId = act.jobId;
      events.push({ type: 'job taken', jobId: act.jobId });
      break;
    }
    case 'work': {
      if (s.jobId === null) return fail('NO JOB');
      const job = JOBS[s.jobId];
      if (s.energy < job.energyCost) return fail('NOT_ENOUGH_ENERGY');
      s.energy = Math.round((s.energy - job.energyCost) * 100) / 100;
      s.fcfa += job.wageFcfa;
      events.push({ type: 'worked', earnedFcfa: job.wageFcfa, energySpent: job.energyCost });
      break;
    }
    case 'buy meal': {
      if (s.energy >= ENERGY_MAX) return fail('ENERGY FULL');
      if (s.fcfa < MEAL.costFcfa) return fail('NOT ENOUGH FCFA');
      const gained = Math.min(MEAL.energy, ENERGY_MAX - s.energy);
      s.fcfa -= MEAL.costFcfa;
      s.energy = Math.round((s.energy + gained) * 100) / 100;
      events.push({ type: 'meal bought', costFcfa: MEAL.costFcfa, energyGained: gained });
      break;
    }
    case 'pay arrears': {
      if (s.rentArrearsFcfa === 0) return fail('NOTHING_OWED');
      const pay = Math.min(act.amountFcfa, s.rentArrearsFcfa);
      if (s.fcfa < pay) return fail('NOT_ENOUGH_FCFA');
      s.fcfa -= pay;
      s.rentArrearsFcfa -= pay;
      events.push({ type: 'arrears paid', paidFcfa: pay, remainingFcfa: s.rentArrearsFcfa });
      break;
    }
    case 'move housing': {
      if (act.tier === s.housingTier) return fail('SAME HOUSING');
      if (s.rentArrearsFcfa > 0) return fail('HAS ARREARS');
      const deposit = HOUSING_TIERS[act.tier].depositFcfa;
      if (s.fcfa < deposit) return fail('NOT ENOUGH FCFA');
      s.fcfa -= deposit;
      s.housingTier = act.tier;
      s.nextRentDueAt = now + RENT_INTERVAL_MS;
      events.push({ type: 'moved', tier: act.tier, depositFcfa: deposit });
      break;
    }
  }

  s.version = state.version + 1;
  return { ok: true, state: s, events };
}
```

END FILE: src/game/engine.ts

src/game/index.ts

BEGIN FILE: src/game/index.ts

```
export * from './constants';
export * from './types';
export { accrue } from './accrue';
export { applyAction } from './engine';
export { createPlayer } from './player';
export type { CreatePlayerInput } from './player';
export { actionSchema, createPlayerSchema } from './schemas';
```

END FILE: src/game/index.ts

Messages and content

src/messages/en.json

BEGIN FILE: src/messages/en.json

```
{
  "app": { "name": "KmerLife" },
  "job": {
    "benskin_rider": "Benskin rider",
    "market trader": "Market trader",
    "tailor": "Tailor",
    "momo agent": "Mobile money agent",
    "private_tutor": "Private tutor"
  },
  "housing": {
    "relatives": "Staying with relatives",
    "chambre": "Rented room",
    "studio": "Studio"
  },
  "venue": {
    "mokoloMarket": "Mokolo Market",
    "marcheCentral": "Central Market",
    "marcheMfoundi": "Mfoundi Market",
    "bastosQuarter": "Bastos",
    "mvogMbiQuarter": "Mvog-Mbi",
    "mvanBusStation": "Mvan Bus Station",
    "lacMunicipal": "Municipal Lake",
    "ngoaEkelleCampus": "Ngoa-Ekelle Campus"
  },
  "error": {
    "INVALID ACTION": "That action is not valid.",
    "NO_JOB": "You need a job first.",
    "SAME_JOB": "You already have this job.",
    "NOT ENOUGH ENERGY": "You are too tired. Rest or eat.",
    "NOT ENOUGH FCFA": "Not enough FCFA.",
    "ENERGY_FULL": "You are not hungry.",
    "NOTHING OWED": "You owe no rent.",
    "HAS ARREARS": "Pay your rent arrears first.",
    "SAME HOUSING": "You already live here."
  }
}
```

END FILE: src/messages/en.json

src/messages/fr.json

BEGIN FILE: src/messages/fr.json

```
{
  "app": { "name": "KmerLife" },
  "job": {
    "benskin rider": "Conducteur de moto-taxi",
    "market trader": "Commerçant(e) de marché",
    "tailor": "Couturier(ère)",
    "momo agent": "Agent de transfert d'argent mobile",
    "private tutor": "Répétiteur(trice)"
  },
  "housing": {
    "relatives": "Chez des proches",
    "chambre": "Chambre en location",
    "studio": "Studio"
  },
  "venue": {
    "mokoloMarket": "Marché Mokolo",
    "marcheCentral": "Marché Central",
    "marcheMfoundi": "Marché Mfoundi",
    "bastosQuarter": "Bastos",
    "mvogMbiQuarter": "Mvog-Mbi",
    "mvanBusStation": "Gare routière de Mvan",
    "lacMunicipal": "Lac Municipal",
    "ngoaEkelleCampus": "Campus de Ngoa-Ekellé"
  },
  "error": {
    "INVALID ACTION": "Cette action n'est pas valide.",
    "NO JOB": "Il te faut d'abord un travail.",
    "SAME JOB": "Tu as déjà ce travail.",
    "NOT_ENOUGH_ENERGY": "Tu es trop fatigué(e). Repose-toi ou mange.",
    "NOT ENOUGH FCFA": "Pas assez de FCFA.",
    "ENERGY_FULL": "Tu n'as pas faim.",
    "NOTHING OWED": "Tu ne dois aucun loyer.",
    "HAS_ARREARS": "Paie d'abord tes arriérés de loyer.",
    "SAME HOUSING": "Tu habites déjà ici."
  }
}
```

END FILE: src/messages/fr.json

content/venues.yaounde.json

BEGIN FILE: content/venues.yaounde.json

```
{
  "city": "yaounde",
  "venues": [
    { "id": "mokolo-market", "nameKey": "venue.mokoloMarket", "type": "market", "actions": ["work",
"buy_meal"], "open": "06:00", "close": "19:00" },
    { "id": "marche-central", "nameKey": "venue.marcheCentral", "type": "market", "actions":
["work", "buy_meal"], "open": "06:00", "close": "19:00" },
    { "id": "marche-mfoundi", "nameKey": "venue.marcheMfoundi", "type": "market", "actions":
["work", "buy_meal"], "open": "06:00", "close": "18:00" },
    { "id": "bastos-quarter", "nameKey": "venue.bastosQuarter", "type": "residential", "actions":
["move_housing", "buy_meal"], "open": "00:00", "close": "24:00" },
    { "id": "mvog-mbi-quarter", "nameKey": "venue.mvogMbiQuarter", "type": "residential",
"actions": ["move housing", "buy meal"], "open": "00:00", "close": "24:00" },
    { "id": "mvan-bus-station", "nameKey": "venue.mvanBusStation", "type": "transport", "actions":
["travel"], "open": "05:00", "close": "21:00" },
    { "id": "lac-municipal", "nameKey": "venue.lacMunicipal", "type": "leisure", "actions":
["chat", "rest"], "open": "00:00", "close": "24:00" },
    { "id": "ngoa-ekelle-campus", "nameKey": "venue.ngoaEkelleCampus", "type": "campus", "actions":
["work", "chat"], "open": "07:00", "close": "18:00" }
  ]
}
```

END FILE: content/venues.yaounde.json

Scripts

scripts/simulate.ts

BEGIN FILE: scripts/simulate.ts

```
/**
 * Headless economy simulator.
 * Usage: npm run simulate -- --players 500 --days 60 --seed 7
 *
 * Runs bots with different play styles through the REAL rules (applyAction) and prints the
 * resulting wealth distribution. Use it to tune src/game/constants.ts BEFORE building UI.
 */
import {
  HOUSING_TIERS,
  JOB_IDS,
  MEAL,
  MS_PER_DAY,
  MS_PER_HOUR,
  applyAction,
  createPlayer,
  type GameAction,
  type JobId,
  type PlayerState,
} from '../src/game';

interface Strategy {
  name: string;
  loginsPerDay: number;
  eatBelowEnergy: number;
  upgrades: boolean;
}

const STRATEGIES: Strategy[] = [
  { name: 'grinder', loginsPerDay: 6, eatBelowEnergy: 30, upgrades: true },
  { name: 'regular', loginsPerDay: 2, eatBelowEnergy: 20, upgrades: true },
  { name: 'casual', loginsPerDay: 1, eatBelowEnergy: 0, upgrades: false },
  { name: 'spender', loginsPerDay: 2, eatBelowEnergy: 70, upgrades: false },
];

function arg(name: string, fallback: number): number {
  const i = process.argv.indexOf(`--${name}`);
  const raw = i >= 0 ? process.argv[i + 1] : undefined;
  const v = raw === undefined ? NaN : Number(raw);
  return Number.isFinite(v) ? v : fallback;
}

/** Small deterministic PRNG so runs are reproducible. */
function mulberry32(seed: number): () => number {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
```

```
  };
}

const START = 1_800_000_000_000;
let keyCounter = 0;
const nextKey = (): string => `sim-${String(++keyCounter).padStart(10, '0')}`;

function step(state: PlayerState, action: GameAction, now: number): PlayerState {
  const r = applyAction(state, action, now);
  return r.ok ? r.state : state;
}

function play(strategy: Strategy, rand: () => number, days: number, id: string): PlayerState {
  const origin = rand() < 0.5 ? 'privileged' : 'struggling';
  let s = createPlayer({ id, name: `Bot ${id}`, origin, city: 'yaounde' }, START);
  const job = JOB_IDS[Math.floor(rand() * JOB_IDS.length)] as JobId;
  s = step(s, { type: 'take_job', key: nextKey(), jobId: job }, START);

  for (let d = 0; d < days; d++) {
    for (let l = 0; l < strategy.loginsPerDay; l++) {
      const now =
        START + d * MS_PER_DAY + Math.floor((l * 16 * MS_PER_HOUR) / strategy.loginsPerDay) + 7 *
MS PER HOUR;

      if (s.rentArrearsFcfa > 0) {
        s = step(s, { type: 'pay arrears', key: nextKey(), amountFcfa: Math.min(s.fcfa,
s.rentArrearsFcfa) || 1 }, now);
      }
      // work until exhausted, eating when allowed
      for (let guard = 0; guard < 40; guard++) {
        const before = s;
        s = step(s, { type: 'work', key: nextKey(), }, now);
        if (s === before) {
          if (s.energy < strategy.eatBelowEnergy || (strategy.eatBelowEnergy > 0 && guard === 0)) {
            const afterEat = step(s, { type: 'buy meal', key: nextKey() }, now);
            if (afterEat === s) break;
            s = afterEat;
          } else break;
        }
      }
      if (strategy.upgrades && s.rentArrearsFcfa === 0) {
        const next = (s.housingTier + 1) as 1 | 2;
        if (s.housingTier < 2) {
          const need = HOUSING TIERS[next].depositFcfa + 3 * HOUSING TIERS[next].rentFcfa;
          if (s.fcfa >= need) s = step(s, { type: 'move housing', key: nextKey(), tier: next },
now);
        }
      }
    }
  }
  // final accrual at the end of the horizon
  const end = START + days * MS PER DAY;
  const r = applyAction(s, { type: 'take job', key: nextKey(), jobId: s.jobId ?? 'tailor' }, end);
  return r.ok ? r.state : s;
}

function pct(sorted: number[], p: number): number {
  if (sorted.length === 0) return 0;
  const idx = Math.min(sorted.length - 1, Math.floor((p / 100) * sorted.length));
  return sorted[idx] as number;
}

function main(): void {
  const players = arg('players', 400);
  const days = arg('days', 60);
  const seed = arg('seed', 1);
  const rand = mulberry32(seed);

  console.log(`KmerLife economy simulation: ${players} players x ${days} days (seed ${seed})`);
  console.log(`Meal: ${MEAL.costFcfa} FCFA -> ${MEAL.energy} energy\n`);
  console.log('strategy  n    p10 FCFA   median     p90        evicted%  avg FCFA/day');

  for (const strat of STRATEGIES) {
    const n = Math.max(1, Math.floor(players / STRATEGIES.length));
    const finals: PlayerState[] = [];
    for (let i = 0; i < n; i++) finals.push(play(strat, rand, days, `${strat.name}-${i}`));
    const wealth = finals.map((f) => f.fcfa - f.rentArrearsFcfa).sort((a, b) => a - b);
    const evicted = finals.filter((f) => f.housingTier === 0 && f.origin === 'privileged').length;
    const avg = wealth.reduce((a, b) => a + b, 0) / wealth.length;
    console.log(
      [
        strat.name.padEnd(9),
```

```
        String(n).padEnd(4),
        String(pct(wealth, 10)).padEnd(10),
        String(pct(wealth, 50)).padEnd(10),
        String(pct(wealth, 90)).padEnd(10),
        `${((evicted / n) * 100).toFixed(0)}%`.padEnd(9),
        Math.round(avg / days),
      ].join(' '),
    );
  }
  console.log('\nTargets are in docs/ECONOMY.md. Tune src/game/constants.ts, rerun, repeat.');
}

main();
```

END FILE: scripts/simulate.ts

scripts/bootstrap-github.mjs

BEGIN FILE: scripts/bootstrap-github.mjs

```
#!/usr/bin/env node
/**
 * Seeds GitHub labels, milestones, issues and branch protection for KmerLife.
 * Requires the GitHub CLI (`gh`), already authenticated (`gh auth status`).
 * Safe to re-run: labels are upserted, existing milestones/issues are skipped.
 *
 * Usage:
 *   node scripts/bootstrap-github.mjs --repo OWNER/NAME [--dry-run] [--skip-protection]
 */
import { execFileSync } from 'node:child_process';
import { readFileSync } from 'node:fs';

const args = process.argv.slice(2);
const flag = (name) => args.includes(`--${name}`);
const value = (name) => {
  const i = args.indexOf(`--${name}`);
  return i >= 0 ? args[i + 1] : undefined;
};

const repo = value('repo');
const dry = flag('dry-run');

if (!repo || !/^[\w.-]+\/[\w.-]+$/.test(repo)) {
  console.error('Usage: node scripts/bootstrap-github.mjs --repo OWNER/NAME [--dry-run] [--skip-
protection]');
  process.exit(1);
}

function gh(ghArgs, input) {
  if (dry) {
    console.log(`[dry-run] gh ${ghArgs.join(' ')}`);
    return '';
  }
  return execFileSync('gh', ghArgs, { input, encoding: 'utf8', stdio: ['pipe', 'pipe', 'pipe'] });
}

const LABELS = [
  ['dev-01', '1D76DB', 'Owner: DEV-01 Kernel and Economy'],
  ['dev-02', '0052CC', 'Owner: DEV-02 Character, Jobs and Housing'],
  ['dev-03', '006B75', 'Owner: DEV-03 City, Travel and Venues'],
  ['dev-04', '5319E7', 'Owner: DEV-04 Platform (auth, database, deploy)'],
  ['dev-05', 'B60205', 'Owner: DEV-05 Social and Realtime'],
  ['dev-06', 'E99695', 'Owner: DEV-06 World, Events and Seasons'],
  ['type:feature', '0E8A16', 'New functionality'],
  ['type:bug', 'D73A4A', 'Something is broken'],
  ['type:balance', 'FBCA04', 'Economy or difficulty tuning'],
  ['type:content', 'C5DEF5', 'Game content (jobs, venues, events, text)'],
  ['type:infra', 'BFD4F2', 'Tooling, CI, deploy, security'],
];

const MILESTONES = [
  'Week 1: Foundations',
  'Week 2: Economy and Content',
  'Week 3: City and Deploy',
  'Week 4: Multiplayer',
  'Week 5: Culture and Pilot Prep',
  'Week 6: Pilot',
];

console.log('== Labels');
for (const [name, color, description] of LABELS) {
  gh(['label', 'create', name, '--repo', repo, '--color', color, '--description', description, '--
force']);
```

```
}

console.log('== Milestones');
for (const title of MILESTONES) {
  try {
    gh(['api', `repos/${repo}/milestones`, '-f', `title=${title}`]);
  } catch (err) {
    const msg = String(err?.stderr ?? err?.message ?? err);
    if (msg.includes('already_exists')) console.log(`milestone exists: ${title}`);
    else throw err;
  }
}

console.log('== Issues');
const issues = JSON.parse(readFileSync(new URL('./issues.json', import.meta.url), 'utf8'));
let existingTitles = new Set();
if (!dry) {
  const out = gh(['issue', 'list', '--repo', repo, '--state', 'all', '--limit', '500', '--json',
'title']);
  existingTitles = new Set(JSON.parse(out).map((i) => i.title));
}
for (const issue of issues) {
  if (existingTitles.has(issue.title)) {
    console.log(`issue exists: ${issue.title}`);
    continue;
  }
  const ghArgs = ['issue', 'create', '--repo', repo, '--title', issue.title, '--body', issue.body];
  for (const label of issue.labels) ghArgs.push('--label', label);
  ghArgs.push('--milestone', issue.milestone);
  gh(ghArgs);
}

if (flag('skip-protection')) {
  console.log('== Branch protection skipped (--skip-protection)');
} else {
  console.log('== Branch protection on main');
  const protection = {
    required status checks: { strict: true, contexts: ['validate'] },
    enforce_admins: false,
    required pull request reviews: {
      required approving review count: 1,
      require code owner reviews: true,
      dismiss_stale_reviews: true,
    },
    restrictions: null,
  };
  try {
    gh(['api', '-X', 'PUT', `repos/${repo}/branches/main/protection`, '--input', '-'],
JSON.stringify(protection));
    console.log('branch protection applied');
  } catch (err) {
    const msg = String(err?.stderr ?? err?.message ?? err);
    console.error('BRANCH PROTECTION NOT APPLIED. GitHub said:\n' + msg);
    console.error(
      'Likely cause: private repos on a free personal plan cannot use branch protection. ' +
        'Options: GitHub Pro (free via the Student Developer Pack), move the repo to an
organization on a paid plan, ' +
        'or make the repo public. Report this to the user; do not work around it silently.',
    );
    process.exitCode = 2;
  }
}
console.log('Done.');
```

END FILE: scripts/bootstrap-github.mjs

scripts/issues.json

BEGIN FILE: scripts/issues.json

```
[
  {
    "title": "[DEV-04] Scaffold Next.js (App Router) into the existing repo",
    "labels": [
      "dev-04",
      "type:infra"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-04\n\n**Goal:** Add Next.js, React, Tailwind and next-intl on top of
the existing structure without moving or deleting anything.\n\n**Acceptance criteria:**\n-
`src/app/[locale]/...` routes exist with en and fr\n- Existing `npm run lint`, `typecheck`, `test`
still pass\n- CI also runs `next build`\n- No file under `src/game/` is modified\n\nRead
`AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
```

```
  {
    "title": "[DEV-04] Supabase project, migrations and RLS",
    "labels": [
      "dev-04",
      "type:infra"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-04\n\n**Goal:** Create tables characters, action log, transactions,
world state with row-level security.\n\n**Acceptance criteria:**\n- Migrations are reversible and
live in `supabase/migrations/`\n- `action log` has UNIQUE (player id, key)\n- RLS: a player can
read only their own character; no client writes to money or energy\n- Types generated into
`src/lib/database.types.ts`\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work
only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-04] Auth and create-character endpoint",
    "labels": [
      "dev-04",
      "type:feature"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-04\n\n**Goal:** Email magic-link sign-in and an API route that calls
`createPlayer` and stores the result.\n\n**Acceptance criteria:**\n- Only MVP cities accepted
(server-side check)\n- Name validation comes from `createPlayerSchema`\n- Never trusts client-
supplied money, energy or tier\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first.
Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Server action handler: lock, reduce, persist",
    "labels": [
      "dev-01",
      "type:feature"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-01\n\n**Goal:** Implement `src/server/handle-action.ts`: load player
FOR UPDATE, call `applyAction`, persist state and action log in one transaction.\n\n**Acceptance
criteria:**\n- Repeating the same idempotency key returns the stored result and changes nothing\n-
Concurrent actions on one player serialize correctly (test with parallel calls)\n- Failures return
the error code from `ErrorCode` unchanged\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md`
first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-02] Character creation UI (FR/EN)",
    "labels": [
      "dev-02",
      "type:feature"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-02\n\n**Goal:** Mobile-first creation flow: name, origin, city (Yaoundé
only).\n\n**Acceptance criteria:**\n- All strings come from `src/messages/{en,fr}.json`\n- Works on
a 360px-wide screen and a slow connection\n- Calls DEV-04's endpoint; no game rules in the
component\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your
owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-02] Dashboard: balance, energy, job picker, work button",
    "labels": [
      "dev-02",
      "type:feature"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-02\n\n**Goal:** Show FCFA and energy, choose a job, press
work.\n\n**Acceptance criteria:**\n- Energy bar predicted locally with `accrue` so it moves without
polling\n- Every action sends a fresh UUID idempotency key\n- Server response always replaces local
state\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned
paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Kernel hardening: property tests",
    "labels": [
      "dev-01",
      "type:infra"
    ],
    "milestone": "Week 1: Foundations",
    "body": "**Owner:** DEV-01\n\n**Goal:** Add property-based tests (fast-check) for accrue and
applyAction.\n\n**Acceptance criteria:**\n- Money is always an integer and never negative\n- Energy
always within 0..ENERGY_MAX\n- accrue is idempotent and monotonic in time\n\nRead `AGENTS.md`,
`CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Economy simulator v2 and tuning pass",
    "labels": [
      "dev-01",
      "type:balance"
```

```
    ],
    "milestone": "Week 2: Economy and Content",
    "body": "**Owner:** DEV-01\n\n**Goal:** Extend `scripts/simulate.ts` (CSV output, more
strategies) and tune constants to the targets in `docs/ECONOMY.md`.\n\n**Acceptance criteria:**\n-
`docs/ECONOMY.md` updated with the new baseline table\n- Inflation target met for casual, regular
and grinder strategies\n- Every constant change is justified in the PR description with simulator
output\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned
paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Design and add money sinks: transport, living costs, upgrades",
    "labels": [
      "dev-01",
      "type:balance"
    ],
    "milestone": "Week 2: Economy and Content",
    "body": "**Owner:** DEV-01\n\n**Goal:** Design first (short doc in `docs/decisions/`), then
implement sinks so a regular player cannot just hoard FCFA.\n\n**Acceptance criteria:**\n- Design
approved by DEV-01 before code\n- Simulator shows savings rate within target\n- New actions added
to `actionSchema` and tests\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work
only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-02] Housing UI and rent arrears flow",
    "labels": [
      "dev-02",
      "type:feature"
    ],
    "milestone": "Week 2: Economy and Content",
    "body": "**Owner:** DEV-02\n\n**Goal:** Show tier, next rent date, arrears, pay arrears, move
housing.\n\n**Acceptance criteria:**\n- Eviction shown clearly with a recovery path\n- All amounts
formatted FCFA with thin-space thousands separators\n\nRead `AGENTS.md`, `CONVENTIONS.md` and
`interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-03] Venue content schema and eight Yaoundé venues",
    "labels": [
      "dev-03",
      "type:content"
    ],
    "milestone": "Week 2: Economy and Content",
    "body": "**Owner:** DEV-03\n\n**Goal:** Define the venue schema in `content/` and fill eight
venues.\n\n**Acceptance criteria:**\n- IDs are stable kebab-case; names are message keys\n- Each
venue lists allowed actions and opening hours\n- Validated by a unit test\n\nRead `AGENTS.md`,
`CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-06] Event content schema and first ten radio trottoir rumors",
    "labels": [
      "dev-06",
      "type:content"
    ],
    "milestone": "Week 2: Economy and Content",
    "body": "**Owner:** DEV-06\n\n**Goal:** Define the event schema and write ten rumors, some true
and some false, in FR and EN.\n\n**Acceptance criteria:**\n- Each rumor has truth flag, effect on
the economy if true, expiry\n- Generated or hand-written text reviewed for tone\n- No real people
or real brands named\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only
inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-03] City map UI",
    "labels": [
      "dev-03",
      "type:feature"
    ],
    "milestone": "Week 3: City and Deploy",
    "body": "**Owner:** DEV-03\n\n**Goal:** Static, mobile-first map of Yaoundé with the eight
venues.\n\n**Acceptance criteria:**\n- Light assets (target under 300 KB total)\n- Tap a venue to
see what you can do there\n- No real-time dependency yet\n\nRead `AGENTS.md`, `CONVENTIONS.md` and
`interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-03] Travel module: on foot, benskin, taxi, car",
    "labels": [
      "dev-03",
      "type:feature"
    ],
    "milestone": "Week 3: City and Deploy",
    "body": "**Owner:** DEV-03\n\n**Goal:** Design and implement travel as a pure module in
`src/game/modules/travel/`.\n\n**Acceptance criteria:**\n- Each mode has time and FCFA cost from
constants\n- Tests in `tests/game/`\n- Registered in the engine by DEV-01 via a small PR\n\nRead
`AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
```

```
  {
    "title": "[DEV-04] Deploy: Vercel plus Supabase, preview environments",
    "labels": [
      "dev-04",
      "type:infra"
    ],
    "milestone": "Week 3: City and Deploy",
    "body": "**Owner:** DEV-04\n\n**Goal:** Production and preview deployments with env vars
documented in `.env.example`.\n\n**Acceptance criteria:**\n- Secrets only in Vercel/Supabase, never
in git\n- Service-role key is server-only\n- Cost alerts configured on every paid service\n\nRead
`AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-05] Ably token endpoint and venue presence",
    "labels": [
      "dev-05",
      "type:feature"
    ],
    "milestone": "Week 4: Multiplayer",
    "body": "**Owner:** DEV-05\n\n**Goal:** `/api/ably-token` issuing scoped tokens, and presence
per venue.\n\n**Acceptance criteria:**\n- Clients never see the Ably API key\n- A token only allows
subscribing to the player's own city\n- Realtime sits behind a `RealtimeTransport` interface so it
can be swapped\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside
your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-05] City chat with report and mute",
    "labels": [
      "dev-05",
      "type:feature"
    ],
    "milestone": "Week 4: Multiplayer",
    "body": "**Owner:** DEV-05\n\n**Goal:** Per-city chat with moderation basics.\n\n**Acceptance
criteria:**\n- Rate limit per player\n- Report and mute work\n- Profanity and abuse policy
documented in `docs/GAME DESIGN.md`\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md`
first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-05] Offline action queue with retry",
    "labels": [
      "dev-05",
      "type:feature"
    ],
    "milestone": "Week 4: Multiplayer",
    "body": "**Owner:** DEV-05\n\n**Goal:** Queue actions locally and replay with the same
idempotency keys when the connection returns.\n\n**Acceptance criteria:**\n- Works across page
reloads\n- Duplicates impossible by construction\n- Clear UI state for pending actions\n\nRead
`AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-05] Reminders: Telegram bot and web push",
    "labels": [
      "dev-05",
      "type:feature"
    ],
    "milestone": "Week 4: Multiplayer",
    "body": "**Owner:** DEV-05\n\n**Goal:** Notify players when rent is due or energy is
full.\n\n**Acceptance criteria:**\n- Opt-in only\n- No personal data stored beyond what is
needed\n- Messages localized\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work
only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-05] Njangi module",
    "labels": [
      "dev-05",
      "type:feature"
    ],
    "milestone": "Week 5: Culture and Pilot Prep",
    "body": "**Owner:** DEV-05\n\n**Goal:** Rotating savings groups as specified in
`docs/GAME DESIGN.md`.\n\n**Acceptance criteria:**\n- Pure rules module with tests including
default and dropout cases\n- Server-authoritative; no client-side money movement\n\nRead
`AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see
`.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-06] Radio trottoir events engine and Eneo power cuts",
    "labels": [
      "dev-06",
      "type:feature"
    ],
    "milestone": "Week 5: Culture and Pilot Prep",
    "body": "**Owner:** DEV-06\n\n**Goal:** Scheduled generation of rumors and outages that affect
gameplay.\n\n**Acceptance criteria:**\n- Idempotent scheduled job\n- True and false rumors affect
```

```
the economy as defined\n- Events delivered through DEV-05's realtime layer\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-06] Echoes: record and replay",
    "labels": [
      "dev-06",
      "type:feature"
    ],
    "milestone": "Week 5: Culture and Pilot Prep",
    "body": "**Owner:** DEV-06\n\n**Goal:** Record anonymized action sequences and replay them as labelled echoes in empty venues.\n\n**Acceptance criteria:**\n- Echoes are clearly labelled in the UI\n- No private data in recordings\n- Never affect real players' economy\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Pilot readiness: analytics, flags, privacy notice",
    "labels": [
      "dev-01",
      "type:infra"
    ],
    "milestone": "Week 5: Culture and Pilot Prep",
    "body": "**Owner:** DEV-01\n\n**Goal:** Event analytics (D1/D7), feature flags, seed data, privacy notice in FR/EN.\n\n**Acceptance criteria:**\n- Privacy notice reviewed against Cameroonian data-protection rules\n- No phone numbers collected unless needed\n- Dashboard for retention\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-01] Run the campus pilot and review the data",
    "labels": [
      "dev-01",
      "type:balance"
    ],
    "milestone": "Week 6: Pilot",
    "body": "**Owner:** DEV-01\n\n**Goal:** Invite a closed cohort, observe, and decide on changes.\n\n**Acceptance criteria:**\n- Written pilot report with D1 and D7 retention and economy health\n- Top five fixes filed as issues\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-06] Season 0 reset tooling",
    "labels": [
      "dev-06",
      "type:feature"
    ],
    "milestone": "Week 6: Pilot",
    "body": "**Owner:** DEV-06\n\n**Goal:** Tooling to close a season and start the next one fairly.\n\n**Acceptance criteria:**\n- Reset is a documented, tested, reversible-by-backup operation\n- Hall of fame preserved\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  },
  {
    "title": "[DEV-04] Security review: RLS tests and secrets audit",
    "labels": [
      "dev-04",
      "type:infra"
    ],
    "milestone": "Week 6: Pilot",
    "body": "**Owner:** DEV-04\n\n**Goal:** Automated tests proving a player cannot read or write another player's data or money.\n\n**Acceptance criteria:**\n- Tests run in CI against a local Supabase\n- Secrets scanning enabled on the repo\n\nRead `AGENTS.md`, `CONVENTIONS.md` and `interfaces.md` first. Work only inside your owned paths (see `.github/CODEOWNERS`)."
  }
]
```

END FILE: scripts/issues.json

Tests

tests/game/engine.test.ts

BEGIN FILE: tests/game/engine.test.ts

```
import { describe, expect, it } from 'vitest';
import {
  ENERGY_MAX,
  EVICTION_ARREARS_PERIODS,
  HOUSING_TIERS,
  JOBS,
  MEAL,
  MS_PER_DAY,
  MS_PER_HOUR,
```

```
  MS_PER_WEEK,
  accrue,
  applyAction,
  createPlayer,
  type GameAction,
  type PlayerState,
} from '../../src/game';

const T0 = 1_800_000_000_000;
let n = 0;
const k = (): string => `key-${String(++n).padStart(8, '0')}`;

const fresh = (origin: 'privileged' | 'struggling' = 'struggling'): PlayerState =>
  createPlayer({ id: 'p1', name: 'Niels', origin, city: 'yaounde' }, T0);

const ok = (state: PlayerState, action: GameAction, now: number): PlayerState => {
  const r = applyAction(state, action, now);
  if (!r.ok) throw new Error(`expected ok, got ${r.code}`);
  return r.state;
};

describe('createPlayer', () => {
  it('privileged starts richer and housed; struggling starts broke with relatives', () => {
    const rich = fresh('privileged');
    const poor = fresh('struggling');
    expect(rich.fcfa).toBeGreaterThan(poor.fcfa);
    expect(rich.housingTier).toBe(1);
    expect(poor.housingTier).toBe(0);
    expect(Number.isInteger(rich.fcfa)).toBe(true);
  });

  it('rejects closed cities and bad names', () => {
    expect(() => createPlayer({ id: 'x', name: 'Ab', origin: 'struggling', city: 'douala' },
T0)).toThrow();
    expect(() => createPlayer({ id: 'x', name: 'A', origin: 'struggling', city: 'yaounde' },
T0)).toThrow();
  });
});

describe('accrue', () => {
  it('regenerates energy over time and caps at ENERGY MAX', () => {
    const s = { ...fresh(), energy: 10 };
    const later = accrue(s, T0 + 2 * MS PER HOUR).state;
    expect(later.energy).toBeGreaterThan(10);
    const muchLater = accrue(s, T0 + 30 * MS_PER_DAY).state;
    expect(muchLater.energy).toBe(ENERGY MAX);
  });

  it('is idempotent for the same instant', () => {
    const s = fresh('privileged');
    const t = T0 + 3 * MS PER WEEK;
    const once = accrue(s, t).state;
    const twice = accrue(once, t).state;
    expect(twice).toEqual(once);
  });

  it('does not go backwards if the clock is behind', () => {
    const s = fresh();
    const r = accrue(s, T0 - 1000);
    expect(r.state).toBe(s);
    expect(r.events).toEqual([]);
  });

  it('charges rent from balance and records arrears when short', () => {
    const s: PlayerState = { ...fresh('privileged'), fcfa: 4_000 }; // tier 1, rent 10 000
    const r = accrue(s, T0 + MS PER WEEK + 1);
    expect(r.state.fcfa).toBe(0);
    expect(r.state.rentArrearsFcfa).toBe(HOUSING TIERS[1].rentFcfa - 4 000);
    expect(r.events.some((e) => e.type === 'rent_charged')).toBe(true);
  });

  it('evicts to tier 0 once arrears reach the threshold', () => {
    const s: PlayerState = { ...fresh('privileged'), fcfa: 0 };
    const r = accrue(s, T0 + (EVICTION ARREARS PERIODS + 1) * MS PER WEEK);
    expect(r.state.housingTier).toBe(0);
    expect(r.events.some((e) => e.type === 'evicted')).toBe(true);
  });

  it('caps catch-up so long absences are not infinitely punishing', () => {
    const s: PlayerState = { ...fresh('privileged'), fcfa: 0 };
    const r = accrue(s, T0 + 200 * MS_PER_WEEK);
```

```
    expect(r.state.rentArrearsFcfa).toBeLessThanOrEqual(8 * HOUSING_TIERS[1].rentFcfa);
  });
});

describe('applyAction', () => {
  it('rejects malformed actions', () => {
    expect(applyAction(fresh(), { type: 'fly' }, T0)).toEqual({ ok: false, code: 'INVALID ACTION'
});
    expect(applyAction(fresh(), { type: 'work', key: 'x' }, T0)).toEqual({ ok: false, code:
'INVALID_ACTION' });
  });

  it('work requires a job and energy, then pays the wage', () => {
    const s0 = fresh();
    expect(applyAction(s0, { type: 'work', key: k() }, T0)).toEqual({ ok: false, code: 'NO JOB' });

    const s1 = ok(s0, { type: 'take_job', key: k(), jobId: 'tailor' }, T0);
    const s2 = ok(s1, { type: 'work', key: k() }, T0);
    expect(s2.fcfa).toBe(s1.fcfa + JOBS.tailor.wageFcfa);
    expect(s2.energy).toBe(s1.energy - JOBS.tailor.energyCost);

    const drained: PlayerState = { ...s2, energy: 1 };
    expect(applyAction(drained, { type: 'work', key: k() }, T0)).toEqual({
      ok: false,
      code: 'NOT ENOUGH ENERGY',
    });
  });

  it('meals restore energy for money and refuse waste', () => {
    const tired: PlayerState = { ...fresh('privileged'), energy: 20 };
    const fed = ok(tired, { type: 'buy meal', key: k() }, T0);
    expect(fed.energy).toBe(20 + MEAL.energy);
    expect(fed.fcfa).toBe(tired.fcfa - MEAL.costFcfa);
    expect(applyAction(fresh('privileged'), { type: 'buy meal', key: k() }, T0)).toEqual({
      ok: false,
      code: 'ENERGY FULL',
    });
    const broke: PlayerState = { ...fresh(), energy: 10 };
    expect(applyAction(broke, { type: 'buy meal', key: k() }, T0)).toEqual({
      ok: false,
      code: 'NOT_ENOUGH_FCFA',
    });
  });

  it('moving housing costs a deposit, needs zero arrears, and resets the rent clock', () => {
    const s = fresh('privileged');
    const moved = ok(s, { type: 'move_housing', key: k(), tier: 2 }, T0 + MS_PER_DAY);
    expect(moved.housingTier).toBe(2);
    expect(moved.fcfa).toBe(s.fcfa - HOUSING_TIERS[2].depositFcfa);
    expect(moved.nextRentDueAt).toBe(T0 + MS_PER_DAY + MS_PER_WEEK);
    expect(applyAction(moved, { type: 'move housing', key: k(), tier: 2 }, T0 +
MS_PER_DAY)).toEqual({
      ok: false,
      code: 'SAME HOUSING',
    });
    const indebted: PlayerState = { ...s, rentArrearsFcfa: 5_000 };
    expect(applyAction(indebted, { type: 'move housing', key: k(), tier: 2 }, T0)).toEqual({
      ok: false,
      code: 'HAS ARREARS',
    });
  });

  it('pay arrears pays at most what is owed', () => {
    const s: PlayerState = { ...fresh('privileged'), rentArrearsFcfa: 5 000 };
    const r = applyAction(s, { type: 'pay_arrears', key: k(), amountFcfa: 99_000 }, T0);
    if (!r.ok) throw new Error(r.code);
    expect(r.state.rentArrearsFcfa).toBe(0);
    expect(r.state.fcfa).toBe(s.fcfa - 5 000);
    expect(applyAction(r.state, { type: 'pay arrears', key: k(), amountFcfa: 10 }, T0)).toEqual({
      ok: false,
      code: 'NOTHING_OWED',
    });
  });

  it('is pure: same input gives same output and the input is never mutated', () => {
    const s = Object.freeze(fresh('privileged')) as PlayerState;
    const a: GameAction = { type: 'take_job', key: 'fixed-key-01', jobId: 'momo_agent' };
    const r1 = applyAction(s, a, T0 + 1000);
    const r2 = applyAction(s, a, T0 + 1000);
    expect(r1).toEqual(r2);
    expect(s.jobId).toBeNull();
  });
```

```
  it('increments version only on success', () => {
    const s = fresh();
    const r = applyAction(s, { type: 'take_job', key: k(), jobId: 'tailor' }, T0);
    if (!r.ok) throw new Error(r.code);
    expect(r.state.version).toBe(s.version + 1);
  });
});
```

END FILE: tests/game/engine.test.ts

tests/game/content.test.ts

BEGIN FILE: tests/game/content.test.ts

```
import { readFileSync } from 'node:fs';
import { describe, expect, it } from 'vitest';
import { ERROR_CODES, JOB_IDS } from '../../src/game';

const read = (p: string): unknown => JSON.parse(readFileSync(new URL(p, import.meta.url), 'utf8'));
type Messages = { job: Record<string, string>; error: Record<string, string>; venue: Record<string,
string> };

describe('content and messages stay in sync', () => {
  const en = read('../../src/messages/en.json') as Messages;
  const fr = read('../../src/messages/fr.json') as Messages;

  it('en and fr have identical key sets', () => {
    const keys = (o: unknown, prefix = ''): string[] =>
      Object.entries(o as Record<string, unknown>).flatMap(([k, v]) =>
        typeof v === 'object' && v !== null ? keys(v, `${prefix}${k}.`) : [`${prefix}${k}`],
      );
    expect(keys(fr).sort()).toEqual(keys(en).sort());
  });

  it('every job has a label in both languages', () => {
    for (const id of JOB IDS) {
      expect(en.job[id]).toBeTruthy();
      expect(fr.job[id]).toBeTruthy();
    }
  });

  it('every error code has a message in both languages', () => {
    for (const code of ERROR CODES) {
      expect(en.error[code]).toBeTruthy();
      expect(fr.error[code]).toBeTruthy();
    }
  });

  it('every venue nameKey resolves in both languages', () => {
    const venues = read('../../content/venues.yaounde.json') as { venues: { id: string; nameKey:
string }[] };
    expect(venues.venues.length).toBe(8);
    for (const v of venues.venues) {
      const key = v.nameKey.replace('venue.', '');
      expect(en.venue[key], v.id).toBeTruthy();
      expect(fr.venue[key], v.id).toBeTruthy();
    }
  });
});
```

END FILE: tests/game/content.test.ts

Empty files

Create each of these with no content at all. They exist only so Git keeps the empty folders.

- src/app/.gitkeep
- src/features/.gitkeep
- src/services/.gitkeep
- src/lib/.gitkeep
- src/server/.gitkeep
- supabase/migrations/.gitkeep

Part 9. Verification checklist and final report

Run each command. Replace `OWNER` as before. Every row must match the expected result.

#	Check	Command	Expected
1	File count	git ls-files	49 files including package-lock.json
2	No secrets tracked	git ls-files .env*	Only .env.example
3	Lint	npm run lint	No errors
4	Typecheck	npm run typecheck	No errors
5	Tests	npm test	2 files, 19 tests passed
6	Simulator	npm run simulate -- --players 200 --days 30 --seed 1	Table with 4 strategies
7	No placeholders	git grep -n "_HANDLE"	No output
8	Repository	gh repo view OWNER/kmerlife --json visibility,defaultBranchRef	Visibility as chosen; default branch main
9	CI	gh run list --repo OWNER/kmerlife --limit 1	Latest run completed, success
10	Labels	gh label list --repo OWNER/kmerlife	Includes dev-01 to dev-06 and 5 type: labels
11	Milestones	gh api repos/OWNER/kmerlife/milestones --jq length	6
12	Issues	gh issue list --repo OWNER/kmerlife --state open --limit 100 --json number --jq length	26
13	Branch protection	gh api repos/OWNER/kmerlife/branches/main/protection --jq .required_pull_request_reviews.require_code_owner_reviews	true (or report why not)
14	CODEOWNERS valid	gh api repos/OWNER/kmerlife/codeowners/errors	{"errors":[]} (errors appear until teammates accept invitations)

Final report template

Send the user a message in this shape, filled in honestly:

```
KmerLife bootstrap report

Repository:      https://github.com/OWNER/kmerlife   (private/public)
Local folder:    <path>
Checks:          lint OK | typecheck OK | 19/19 tests | simulator OK | CI green
GitHub seeded:   11 labels, 6 milestones, 26 issues
Branch protection: applied / NOT applied (reason: ...)
CODEOWNERS:      valid / N errors (reason: ...)
Collaborators:   invited <names> / not invited

Needs your action:
 1. <e.g. teammates must accept the collaborator invitation>
 2. <e.g. enable GitHub Pro via the Student Developer Pack to unlock branch protection>
 3. <anything else>

Not done on purpose (see the issue backlog): Next.js scaffold, Supabase, deploy.
```

Part 10. Troubleshooting, decisions and open questions

Troubleshooting

Symptom	Cause and fix
`gh: command not found`	GitHub CLI not installed. User installs it from cli.github.com, runs `gh auth login`, then you resume at step 0.
`npm ci` fails in CI with a missing lock file	`package-lock.json` was not committed. Run `npm install` locally, commit the lock file, push.
Branch protection returns 403 with an upgrade message	Private repository on a free personal plan. Options: GitHub Pro (free through the GitHub Student Developer Pack), an organization on a paid plan, or a public repository. Ask the user; never choose for them.
`gh api .../codeowners/errors` lists errors	A handle is invalid, misspelled, or the user has no Write access yet. Fix the handle or wait for the invitation to be accepted.
Tests fail after you created the files	Transcription error in a file from Part 8. Diff the file against the document. Do not change constants or expected values.
Line-ending warnings on Windows	`.gitattributes` forces LF. If warnings persist, run `git config core.autocrlf input` for this repository.
`gh repo create` says the repository already exists	Ask the user whether to use another name or push to the existing empty repository (`git remote add origin ...` then `git push -u origin main`).

Decisions already made

- Stack: Next.js (App Router), TypeScript strict, Tailwind, Zustand, Supabase, Ably, Vercel, Vitest, zod.
- Architecture: pure rules kernel, server-authoritative, lazy time accrual, idempotency keys, slow multiplayer.
- Team: six developers with vertical feature ownership, enforced by CODEOWNERS.
- Scope: Yaoundé only for the MVP, 5 careers, 8 venues, 3 housing tiers.
- Cameroon-specific systems: njangi, radio trottoir, Eneo power cuts, echoes, seasons.
- Bootstrap deliberately excludes the Next.js scaffold, Supabase project and deployment; they are issues for DEV-04.

Open questions for the team (not for the agent)

- Keep Ably, or use Supabase Realtime to remove one vendor? Both fit behind `RealtimeTransport`.
- Minimum player age before the pilot.
- Njangi details: group discovery, invitations, turn swapping.
- Money sinks: which ones, and the target savings rate (see `docs/ECONOMY.md`).
- Legal review of the privacy notice against current Cameroonian data-protection rules.
- Hosting plan limits: scheduled jobs frequency and realtime message budgets.
- Whether the GitHub account has a plan that supports branch protection on private repositories.